equilibrium

QUANTUS

,

Quantus Network: Security and Performance Review

Draft Report, v0.1
2026-03-20

Equilibrium Group Oy
Meritullinkatu 1
Helsinki
Finland

TABLE OF CONTENTS

0. **EXECUTIVE SUMMARY** 4
1. **PROJECT SUMMARY** 6
   - The Project 6
   - Context and Scope 6
   - Disclaimers 7
   - Remediation 8
2. **MINER REWARDS** 9
   - EQ-QNT-MINING-R-03: Emission Simulation Test Quality Issues 9
   - EQ-QNT-MINING-R-02: Transaction Fee Treasury Fallback Untested 10
   - EQ-QNT-MINING-R-01: Mining-Rewards Test Helpers Are Incorrect and Overly Specific 10
   - EQ-QNT-MINING-R-04: Superfluous `block_lifecycle_works` Test 12
3. **POSEIDON HASH FUNCTION** 13
   - EQ-QNT-ZK-F-01: Poseidon2 `permute_swapped` Ignores the Swap Parameter 13
   - EQ-QNT-ZK-O-03: Poseidon2 Structural Parameters Defined Independently Across Five Files 14
   - EQ-QNT-ZK-O-04: Poseidon2 Permutation Object Recreated on Every CPU Hash Call 15
   - EQ-QNT-ZK-O-07: Poseidon2 Gate Does Not Elide Round-0 S-Box Constraints 16
4. **WORMHOLE** 18
   - EQ-QNT-WORMHOLE-F-02: Unsigned Proof Endpoint Vulnerable to Weight-Exhaustion Spam 18
   - EQ-QNT-WORMHOLE-F-03: Miner Rewards Silently Burned Due to Incorrect Preimage Handling 20
   - EQ-QNT-WORMHOLE-I-01: Mining Rewards Benchmark Confuses Account Address with Preimage 25
   - EQ-QNT-WORMHOLE-F-04: Non-Native Asset Transfers Unsupported Despite Partial Implementation 25
   - EQ-QNT-WORMHOLE-F-05: Transfer Variants Missing from Wormhole Proof Recorder 29
   - EQ-QNT-DOC-F-01: Leftover Amount Redemption Issue in QIP-5 30
   - EQ-QNT-WORMHOLE-R-01: Unused Error Variants in Wormhole Pallet 32
   - EQ-QNT-WORMHOLE-R-02: `volume_fee_bps` Lacks a Single Source of Truth 33
   - EQ-QNT-WORMHOLE-R-04: Dummy Proof Detection Uses Different Conditions Across the Stack 35
   - EQ-QNT-WORMHOLE-R-05: Storage Proof Preprocessing Operates on Hex Strings Instead of Binary Data 38
   - EQ-QNT-WORMHOLE-R-07: `WormholeProver::default()` Silently Falls Back to Fresh Circuit Build on Any File Error 41
   - EQ-QNT-WORMHOLE-R-08: Infallible `record_transfer` Returns `Result` 42
   - EQ-QNT-WORMHOLE-R-06: Inconsistent `UnspendableAccount` Conversion Patterns in Wormhole Example 45
   - EQ-QNT-WORMHOLE-R-09: Wormhole pallet weights are stale 46
   - EQ-QNT-WORMHOLE-O-01: Wormhole Test Coverage 48
   - EQ-QNT-WORMHOLE-O-02: MAX_PROOF_LEN Storage Proof Constraint Undocumented 50
   - EQ-QNT-WORMHOLE-O-03: Dummy Proof Nullifiers Written to Permanent Storage 52
   - EQ-QNT-WORMHOLE-O-04: QIP-5 describes parent_hash-based cross-block aggregation that was never implemented 56
   - EQ-QNT-CHAIN-R-01: Outdated trie-db Dependency Triggers Rust 2024 Edition Warning 58
   - EQ-QNT-WORMHOLE-I-02: Fee Formula Produces Zero Fee at `volume_fee_bps = 10000` 59
5. **OTHER OBSERVATIONS** 61
   - EQ-QNT-MISC-R-01: Unnecessary Use of Clone 61
   - EQ-QNT-MISC-R-03: Overly Broad `pub` Visibility across ZK Circuit Crates 62
   - EQ-QNT-MISC-O-01: Mining Rewards and Wormhole minting accounts are derived differently 64
   - EQ-QNT-MISC-R-02: Unused Runtime Constants 64
   - EQ-QNT-ZK-I-01: NEON SIMD Poseidon Code Not Validated in This Audit 65
   - EQ-QNT-ZK-O-02: Dual Hash Strategy: Poseidon for FRI, Poseidon2 for Application Circuits 66
   - EQ-QNT-ZK-O-05: Verifier Crate Duplicates Gate and Hash Logic from the Prover 66
   - EQ-QNT-ZK-O-06: Protocol-Level Constants Duplicated across Crates without a Single Source of Truth 69
6. **ZK TRIE AND STATE MACHINE** 71
   - EQ-QNT-TRIE-O-02: Compact Proof API Simplified to Passthrough 71
7. **Experiments** 73
   - A. Miner Volume Fee Misdirection 73
   - B. Garbage Proof Block Saturation 73
8. **CLASSIFICATIONS** 75

9. CONCLUSION 76
10. REFERENCES 77
11. APPENDIX A: Poseidon2 Permutation Parameters 79
12. APPENDIX B: Wormhole Nullifier Storage Growth Estimate 81

0. EXECUTIVE SUMMARY

Auditor: Equilibrium Group Oy
Client: Quantus Labs LLC

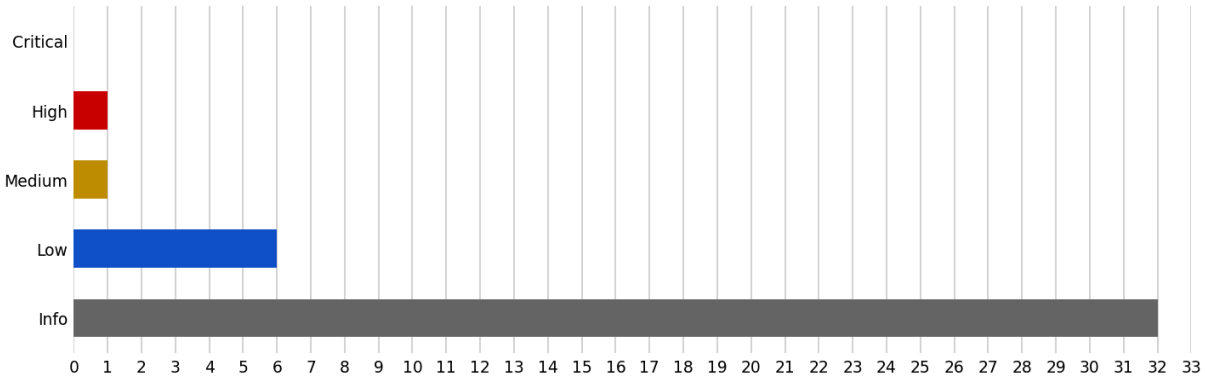
Scope: Security Review of QIP-5: Entangled Addresses, Wormhole Pallet, Mining-Rewards Pallet, and Poseidon2 Hash Implementation.

Equilibrium conducted a security review of the Quantus Network's wormhole pallet and mining-rewards pallet, along with supporting ZK circuits and the Poseidon2 hash implementation for plonky2. The review was conducted remotely by Equilibrium between 2026-01-21 and 2026-03-20, with an emphasis on cryptographic design soundness, zero-knowledge proof correctness, implementation security, and performance.

The review covered the design and implementation of QIP-5′s privacy-preserving transfer scheme, the proof aggregation mechanism, the mining-rewards distribution logic, and a broader refactor of the Quantus plonky2 fork.

KEY FINDINGS:

We gathered 35 findings and observations in the review. No critical issues bearing on security were identified. Review found one high-severity issue in the wormhole pallet's unsigned proof submission endpoint, due to absence of meaningful pre-dispatch validation or rate limiting. We identified one medium-severity issue in miner fee distribution, resulting in volume fee being burned. Additionally, four low-severity findings, and a number of informational observations on code quality, documentation, testing gaps, and architectural decisions were noted.



The Poseidon2 implementation conforms to the specification and is of high quality. qp-plonky2 refactor, while comprehensive, was performed correctly and does not change preexisting

security guarantees. The informational findings centre on maintainability risks from the prover/verifier crate split, inconsistent constant definitions, documentation gaps, and uneven test coverage.

1. PROJECT SUMMARY

This security review assesses the wormhole pallet and the mining-rewards pallet of the Quantus Network blockchain, along with the supporting ZK circuit library and Poseidon2 hash implementation. Our assessment produced findings, observations, and recommendations that are detailed in this report.

The Project

The wormhole pallet implements QIP-5 ("Entangled Addresses"), a privacy-preserving transfer scheme that uses zero-knowledge proofs, nullifiers, and unspendable addresses to let users move funds without revealing the link between sender and receiver. An aggregation mechanism allows batching many individual transfers into a single proof, improving the anonymity set of each transfer and scaling verification better than processing proofs one by one.

The mining-rewards pallet distributes block rewards to miners identified by Poseidon2 hash preimages deposited in block digests as part of the proof-of-work consensus.

Both are Substrate/FRAME pallets integrated into the Quantus chain runtime.

A new Poseidon2 hash implementation for plonky2 underpins both pallets and was reviewed as part of this engagement. A broader refactor of the Quantus plonky2 fork (`qp-plonky2`) was also in scope, covering circuit structure and API changes beyond the hash addition itself.

The `zk-trie` and `zk-state-machine` crates adapt Substrate's storage trie for ZK circuit compatibility. `zk-trie` (a fork of `sp-trie`) rewrites trie node encoding so that every component is padded to 8-byte ("felt") boundaries, enabling efficient verification inside plonky2 circuits operating over the Goldilocks field. `zk-state-machine` (a fork of `sp-state-machine`) is a thin adaptation layer that points its trie dependency at `zk-trie` and disables compact proofs.

Context and Scope

The following repositories and commits were in scope for this review:

- QIP-5: Entangled Addresses
  - https://github.com/Quantus-Network/improvement-proposals
  - Commit: `eff9f5cff92863586ba1aab39f0984e7970fb527` (2026-01-21)
- Quantus Chain (wormhole pallet, mining-rewards pallet, runtime)
  - https://github.com/Quantus-Network/chain
  - Commit: `f838f70dffca6e47e34b3f10a75d607297dcd250` (2026-02-17)
- ZK Circuits (wormhole aggregator, dummy proofs, test helpers)
  - https://github.com/Quantus-Network/qp-zk-circuits

  - Commit: `2246b4e967136ddaa25f41cb878a9aa24950a6ea` (2026-02-17)
- Plonky2 fork (Poseidon2 implementation, plonky2 refactor)
  - https://github.com/Quantus-Network/qp-plonky2
  - Commit: `df2367ff7fe2a4cdec17068ffe84bf21c7f1af99` (2026-02-11)
- ZK Trie (felt-aligned trie encoding for ZK circuits)
  - https://github.com/Quantus-Network/zk-trie
  - Commit: `bb1c142115ef1c7cae9378dd438b179d113d29cb` (2025-06-12)
- ZK State Machine (state machine adaptation for zk-trie)
  - https://github.com/Quantus-Network/zk-state-machine
  - Commit: `06d1e9545428b57aa56e8997c7bf24eaeb48a103` (2025-06-12)

The following items were out of scope:

- Crate dependencies
- Additional systems, integrations, or resources
- Other pallets and runtime modules not listed above

Disclaimers

This audit does not guarantee the absence of vulnerabilities. It is a best-effort, time-boxed analysis intended to identify as many security issues as possible within the agreed scope. Undiscovered vulnerabilities may still exist. We recommend continued security review and testing beyond this engagement.

Remediation

To be completed when fix commits are available.

2. MINER REWARDS

EQ-QNT-MINING-R-03: Emission Simulation Test Quality Issues

`test_emission_simulation_120m_blocks` has the following issues that reduce its value as a correctness check.

Stale assertion message

The remaining-supply assertion checks `< 1.0%` but the error message reads `"<10%"`:

pallets/mining-rewards/src/tests.rs:513-514

```
513 |    let remaining_percentage = ((MaxSupply::get() - current_supply) as f64 / MaxSupply::get() as
f64) * 100.0;
514 |    assert!(remaining_percentage < 1.0, "Should have <10% supply remaining, got {:.2}%",
remaining_percentage);
```

Early exit weakens supply-cap assertion

The simulation breaks when `block_reward < 1000`:

pallets/mining-rewards/src/tests.rs:447-450

```
447 |        // Early exit if rewards become negligible
448 |        if block_reward < 1000 { // Less than 1000 raw units (very small)
449 |          break;
450 |        }
```

This means the assertion `current_supply <= MaxSupply` at line 503 is never stressed — the simulation stops long before supply could approach the cap. The test's stated goal of validating emission behaviour is partially defeated by exiting before the interesting boundary is reached.

Duplicated exit conditions

The nested `while/for` structure duplicates the exit conditions (`current_supply >= MaxSupply`, `block_reward < 1000`) across the outer while-check, inner for-body, and the post-report check. A single loop with a modulo-based reporting interval (`if block % REPORT_INTERVAL == 0`) would consolidate the exit logic into one place.

Recommendation

Fix the assertion message, remove the early exit on small rewards (or continue minting to exhaustion and assert the cap holds), and flatten the loop structure.

EQ-QNT-MINING-R-02: Transaction Fee Treasury Fallback Untested

When no miner is present in the block digest, `mint_reward` is called with `None` for the transaction fees, routing them to the treasury:

pallets/mining-rewards/src/lib.rs:175-176

```
175 |     // Send fees to miner if any
176 |     Self::mint_reward(miner.clone(), tx_fees);
```

The only no-miner test (`rewards_go_to_treasury_when_no_miner`) does not collect any fees before finalization, so only the block reward fallback is exercised. The fee fallback path is never tested.

Recommendation

Add a test that calls `collect_transaction_fees` before `on_finalize` with no miner digest set, and assert the treasury receives both the block reward and the accumulated fees.

EQ-QNT-MINING-R-01: Mining-Rewards Test Helpers Are Incorrect and Overly Specific

Test utility functions for the `mining-rewards` pallet could be improved for correctness and readability. There are the following issues.

Incorrect fallback in `set_miner_digest`

`set_miner_digest` accepts an `AccountId32` and attempts to reverse-lookup the preimage. It only handles two hard-coded addresses; the else branch uses the account's raw bytes as the preimage:

pallets/mining-rewards/src/mock.rs:170-184

```
170 | pub fn set_miner_digest(miner_account: sp_core::crypto::AccountId32) {
171 |  // Find the preimage that corresponds to this miner address
172 |  let preimage = if miner_account == miner() {
173 |    miner_preimage_1()
174 |  } else if miner_account == miner2() {
175 |    miner_preimage_2()
176 |  } else {
177 |    // For other miners, use their raw bytes as preimage for testing
178 |    let mut preimage = [0u8; 32];
179 |    preimage.copy_from_slice(miner_account.as_ref());
180 |    preimage
181 |  };
182 |
183 |  set_miner_preimage_digest(preimage);
184 | }
```

This is incorrect: the pallet hashes the preimage with Poseidon2 to derive the miner address, so depositing the account bytes as a preimage produces a *different* address than the one passed in. Any test using `set_miner_digest` with an address other than `miner()` or `miner2()` would silently reward the wrong account.

Redundant round-trip

Every test call site follows the pattern `set_miner_digest(miner())`, where `miner()` derives an address from a preimage, only for `set_miner_digest` to look up the preimage from the address. The digest only needs the preimage, making the address round-trip unnecessary.

Proliferation of specific helpers

Four helpers exist to support exactly two test accounts:

pallets/mining-rewards/src/mock.rs:137-151

```
137 | pub fn miner_preimage_1() -> [u8; 32] {
138 |  miner_preimage(1)
139 | }
140 |
141 | pub fn miner_preimage_2() -> [u8; 32] {
142 |  miner_preimage(2)
143 | }
144 |
145 | pub fn miner() -> sp_core::crypto::AccountId32 {
146 |  wormhole_address_from_preimage(miner_preimage_1())
147 | }
148 |
149 | pub fn miner2() -> sp_core::crypto::AccountId32 {
150 |  wormhole_address_from_preimage(miner_preimage_2())
151 | }
```

Recommendation

Replace the above with a single struct that derives both preimage and address from an arbitrary numeric id:

```
struct TestMiner(u64);

impl TestMiner {
    fn preimage(&self) -> [u8; 32] {
        let mut buf = [0u8; 32];
        buf[..8].copy_from_slice(&self.0.to_le_bytes());
        buf
    }

    fn account_id(&self) -> AccountId32 {
        wormhole_address_from_preimage(self.preimage())
```

```
    }
}
```

Tests then call `set_miner_preimage_digest(miner.preimage())` directly, removing `set_miner_digest` and the four per-account helpers entirely.

EQ-QNT-MINING-R-04: Superfluous `block_lifecycle_works` Test

The `block_lifecycle_works` test duplicates coverage already provided by `miner_reward_with_transaction_fees_works`. Its only unique assertion is the weight returned by `on_initialize`:

pallets/mining-rewards/src/tests.rs:240-241

```
240 |    let weight = MiningRewards::on_initialize(1);
241 |    assert_eq!(weight, <()>::on_finalize_rewarded_miner());
```

The remaining lines — fee collection, miner reward calculation, and balance check — are identical in purpose to the existing test. Since `on_initialize` has no side effects beyond returning a weight, calling it before `on_finalize` does not exercise any additional code path.

Recommendation

Move the weight assertion into the existing test or into a dedicated one-liner, and remove the rest.

3. POSEIDON HASH FUNCTION

EQ-QNT-ZK-F-01: Poseidon2 `permute_swapped` Ignores the Swap Parameter

The `AlgebraicHasher` trait requires implementations to provide `permute_swapped`, which conditionally swaps two input chunks before permuting. This is used by plonky2's built-in Merkle proof verification gadget to select left/right child ordering. The Poseidon2 implementation accepts the swap parameter but silently discards it:

plonky2/src/hash/poseidon2.rs:169-196

```
169 |     fn permute_swapped<const D: usize>(
170 |         inputs: Self::AlgebraicPermutation,
171 |         _swap: BoolTarget, // still ignored (semantics unchanged)
172 |         b: &mut CircuitBuilder<F, D>,
173 |     ) -> Self::AlgebraicPermutation
174 |     where
175 |         F: RichField + Extendable<D>,
176 |     {
177 |         let gate_type = Poseidon2Gate::<F, D>::new();
178 |         let row = b.add_gate(gate_type, vec![]);
179 |
180 |         let inp = inputs.as_ref();
181 |         assert_eq!(inp.len(), P2_WIDTH);
182 |
183 |         // connect inputs
184 |         for i in 0..P2_WIDTH {
185 |             b.connect(
186 |                 inp[i],
187 |                 Target::wire(row, Poseidon2Gate::<F, D>::wire_input(i)),
188 |             );
189 |         }
190 |
191 |         // collect outputs
192 |         let state: [Target; P2_WIDTH] =
193 |             core::array::from_fn(|i| Target::wire(row, Poseidon2Gate::<F, D>::wire_output(i)));
194 |
195 |         Poseidon2Permutation { state }
196 |     }
```

The original Poseidon gate handles this correctly with a dedicated `WIRE_SWAP` and associated constraints that enforce child ordering inside the proven computation. The Poseidon2 gate has no swap wire, and `permute_swapped` feeds inputs to the gate without conditional reordering.

This function is not called in any production code path. The application circuits that need Merkle path verification (e.g. the voting circuit) implement child ordering externally using `builder.select()` before calling `hash_n_to_hash_no_pad_p2`, bypassing `permute_swapped` entirely.

The only call site is a benchmark (`plonky2/benches/hashing.rs`), which does not verify swap correctness.

The implementation exists solely to satisfy the `AlgebraicHasher` trait bound, which is required for `Poseidon2Hash` to be usable as a type parameter in generic hashing contexts. However, any future code that calls `builder.verify_merkle_proof_to_cap::<Poseidon2Hash>(...)` or otherwise passes `Poseidon2Hash` through the standard Merkle gadget would silently produce a broken circuit. Since the swap is ignored, the circuit always computes `hash(current, sibling)` regardless of the path bit. When the path bit indicates the current node is a right child, the circuit computes a different intermediate hash than the real Merkle tree. This is a *completeness* failure: an honest prover holding a valid Merkle proof with any right-child steps cannot produce a satisfying witness, because the constrained root will not match. It is not a soundness failure — exploiting the wrong ordering to forge a proof would require a preimage attack on Poseidon2.

Recommendation
Implement the swap inside `permute_swapped` using `builder.select()` to conditionally reorder the first `NUM_HASH_OUT_ELTS` inputs with the next `NUM_HASH_OUT_ELTS` before connecting them to the Poseidon2 gate. This matches what the voting circuit already does manually, and removes the trap for future callers. Alternatively, if the standard Merkle gadget is intentionally never used with Poseidon2, remove the `AlgebraicHasher` implementation entirely to make the incompatibility a compile-time error rather than a silent runtime one.

EQ-QNT-ZK-O-03: Poseidon2 Structural Parameters Defined Independently Across Five Files

The Poseidon2 sponge rate, state width, round counts, and internal diagonal matrix are each hardcoded as independent constants in multiple files with no shared source of truth. Changing any parameter in one location without updating all others would silently break the relationship between the gate constraints, the CPU sponge, and the verifier — producing proofs that fail to verify or, worse, a prover and verifier that disagree on the hash function.

The rate alone is defined five times:

File	Constant	Scope
`plonky2/.../gates/poseidon2.rs`	`pub const P2_RATE = 4`	prover gate
`plonky2/.../hash/poseidon2.rs`	`const SPONGE_RATE = 4`	prover sponge
`plonky2/.../hash/hashing.rs`	`const RATE = 4`	circuit builder sponge
`verifier/.../gates/poseidon2.rs`	`pub const P2_RATE = 4`	verifier gate
`verifier/.../hash/poseidon2.rs`	`const SPONGE_RATE = 4`	verifier sponge

The circuit builder's definition (in `hashing.rs`) is a function-local `const` with only a comment — `// must match your gate` — acknowledging the coupling. The width (12), external round count (8), and internal round count (22) are similarly duplicated across prover and verifier crates.

The 12-element internal diagonal (`MATRIX_DIAG_12_GOLDILOCKS`) is hardcoded as literal hex values in both the prover gate and the verifier gate independently. A transcription error in either copy would cause the verifier to reject all valid proofs.

The round constants are the one exception: they are properly centralised in the `qp-poseidon-constants` crate and imported by both prover and verifier.

Recommendation
Centralise Poseidon2 structural parameters (width, rate, round counts, diagonal) alongside the round constants in `qp-poseidon-constants`, and import them in both crates.

EQ-QNT-ZK-O-04: Poseidon2 Permutation Object Recreated on Every CPU Hash Call

The CPU-side Poseidon2 permutation helper rebuilds the `Poseidon2` instance — including all round constants — on every invocation:

plonky2/src/hash/poseidon2.rs:33-50

```
33 | fn p2_permute_gl(mut state: [GL; SPONGE_WIDTH]) -> [GL; SPONGE_WIDTH] {
34 |     // Convert to p3 Goldilocks.
35 |     let mut s_p3 = [P3G::ZERO; SPONGE_WIDTH];
36 |     for i in 0..SPONGE_WIDTH {
37 |         // GL -> u64 -> P3G (both mod 2^64 - 2^32 + 1)
38 |         s_p3[i] = unsafe { P3G::from_canonical_unchecked(state[i].to_canonical_u64()) };
39 |     }
40 |
41 |     let poseidon2 = create_poseidon();
42 |     let mut st = s_p3;
43 |     poseidon2.permute_mut(&mut st);
44 |
45 |     // Back to plonky2 GL
46 |     for i in 0..SPONGE_WIDTH {
47 |         state[i] = GL::from_noncanonical_u64(st[i].as_canonical_u64());
48 |     }
49 |     state
50 | }
```

Since `p2_permute_gl` is called once per sponge block, hashing n field elements at rate 4 triggers $\left\lceil \frac{n+1}{4} \right\rceil$ full reconstructions of the permutation object. The same pattern appears in the verifier crate's copy of this function.

Recommendation

Initialise the `Poseidon2` instance once in a `LazyLock` or `OnceLock` static and reuse it across calls. The instance is determined entirely by compile-time constants and is safe to share.

EQ-QNT-ZK-O-07: Poseidon2 Gate Does Not Elide Round-0 S-Box Constraints

The original Poseidon gate skips constraining intermediate S-box wires for the first full round, because the S-box input at that point is a degree-1 function of the input wires (wire value plus a round constant). Applying the S-box produces a degree-7 constraint, which is within plonky2′s limit. A checkpoint wire is only needed from round 1 onward, where the S-box input is already degree 7 from the previous round's output.

The Poseidon2 gate does not apply this optimization. Its first external round's S-box input is also degree 1 (input wires through the linear MDS preamble plus round constants), yet it allocates and constrains 12 intermediate wires for that round. Removing them would save 12 constraints per permutation gate, reducing the count from 130 to 118 — matching the original Poseidon gate.

The fix wraps the round-0 checkpoint in a conditional, identical to the pattern used by the original Poseidon gate. In `eval_unfiltered` (and its `_circuit` counterpart):

```
 // 2) 4 initial external rounds
 for r in 0..4 {
     for i in 0..P2_WIDTH {
         state[i] += ext_c::<F, D>(ext_init[r][i]);
     }
-    for i in 0..P2_WIDTH {
-        let sbox_in = lw[Self::wire_ext_sbox(ext_round_idx, i)];
-        constr.push(state[i] - sbox_in);
-        state[i] = sbox_in;
-    }
+    // Round 0: state is degree-1 (input wires through linear
+    // MDS preamble + constants), so sbox7 produces degree 7
+    // without a checkpoint.
+    if ext_round_idx != 0 {
+        for i in 0..P2_WIDTH {
+            let sbox_in = lw[Self::wire_ext_sbox(ext_round_idx, i)];
+            constr.push(state[i] - sbox_in);
+            state[i] = sbox_in;
+        }
+    }
     for i in 0..P2_WIDTH {
         state[i] = sbox7_ext::<F, D>(state[i]);
     }
```

And in `num_constraints`:

```
-P2_EXT_ROUNDS * P2_WIDTH + P2_INTERNAL_ROUNDS + P2_WIDTH
-// = 8*12 + 22 + 12 = 130
+(P2_EXT_ROUNDS - 1) * P2_WIDTH + P2_INTERNAL_ROUNDS + P2_WIDTH
+// = 7*12 + 22 + 12 = 118
```

The wire layout and witness generator are unchanged — the round-0 S-box wires become unconstrained dead wires, matching how the original Poseidon gate treats its `wire_full_sbox_0(0, _)` slots.

Recommendation

Apply the `if ext_round_idx != 0` guard shown above to `eval_unfiltered` and its `eval_unfiltered_circuit` counterpart, and update `num_constraints` to use the `(P2_EXT_ROUNDS - 1) * P2_WIDTH` formula. The same change must be applied to the verifier crate's duplicate of the gate.

4. WORMHOLE

EQ-QNT-WORMHOLE-F-02: Unsigned Proof Endpoint Vulnerable to Weight-Exhaustion Spam

The `verify_aggregated_proof` extrinsic is an unsigned call with no meaningful pre-dispatch validation. The `validate_unsigned` implementation only checks that proof bytes are non-empty:

pallets/wormhole/src/lib.rs:467-483

```
467 |     fn validate_unsigned(_source: TransactionSource, call: &Self::Call) -> TransactionValidity {
468 |       match call {
469 |         Call::verify_aggregated_proof { proof_bytes } => {
470 |           // Basic validation: check proof bytes are not empty
471 |           if proof_bytes.is_empty() {
472 |             return InvalidTransaction::Custom(2).into();
473 |           }
474 |           ValidTransaction::with_tag_prefix("WormholeAggregatedVerify")
475 |             .and_provides(sp_io::hashing::blake2_256(proof_bytes))
476 |             .priority(TransactionPriority::MAX)
477 |             .longevity(5)
478 |             .propagate(true)
479 |             .build()
480 |         },
481 |         _ => InvalidTransaction::Call.into(),
482 |       }
483 |     }
```

Any non-empty byte sequence passes validation, receives `TransactionPriority::MAX`, and enters the transaction pool. The `provides` tag is derived from `blake2_256(proof_bytes)`, so each distinct payload gets its own tag with no deduplication across different inputs. Substrate guidance [1] recommends that `validate_unsigned` enforce rate-limiting, deduplication, and replay prevention; this implementation does none of these.

Issue

An attacker can submit arbitrary bytes to `verify_aggregated_proof` at no cost. The extrinsic is unsigned so no fees are charged, and the attacker does not need to generate a valid ZK proof or even hold any tokens.

While invalid proof bytes will fail quickly at deserialization (line 244), Substrate reserves the full declared weight for the block regardless of early failure. The weight declaration is a placeholder that was never benchmarked (see EQ-QNT-WORMHOLE-R-09):

pallets/wormhole/src/lib.rs:233-234

```
233 |    #[pallet::call_index(2)]
234 |    #[pallet::weight(<T as Config>::WeightInfo::verify_aggregated_proof())]
```

The total declared weight, including the storage overhead, is approximately 120 ms:

Component	ref_time (ps)	Duration
Base compute (weights.rs)	115 744 000 000	115.7 ms
reads(32) × 25 µs	800 000 000	0.8 ms
writes(32) × 100 µs	3 200 000 000	3.2 ms
Total declared	**119 744 000 000**	**≈ 119.7 ms**
Normal-dispatch quota (75% × 6 s)	4 500 000 000 000	4 500 ms
Quota consumed per proof	2.66%	
Proofs to saturate quota	37	

By submitting 37 distinct garbage payloads per block period, an attacker can saturate the normal-dispatch quota and crowd out legitimate transactions. Each unique byte sequence creates a separate `provides` tag, so there is no limit on the number of distinct payloads in the pool at any given time.

Recommendations

The core problem is that unsigned transactions bypass Substrate's fee market, which is its primary spam deterrent. The extrinsic is unsigned to preserve the wormhole's privacy guarantees: requiring the recipient to sign and submit their own proof would link their exit address to their network identity, defeating the purpose of anonymous transfers.

Immediate mitigations:

0. **Post-dispatch weight correction.** Return actual consumed weight via `DispatchResultWithPostInfo`. Proofs that fail deserialization complete in microseconds; refunding the unused weight frees block capacity for subsequent extrinsics in the same block. This neutralizes the garbage-bytes attack vector, where arbitrary data monopolizes block weight despite failing immediately.

1. **Move cheap checks before ZK verification.** Public inputs are available after deserialization (line 244). Block hash existence, volume fee rate, and minimum transfer

amount checks should all precede the `verifier.verify()` call. Combined with post-dispatch weight correction, proofs that fail these checks also refund their weight.

Together these two changes cover garbage payloads and policy-violating proofs. The remaining gap is proofs that deserialize, pass all cheap checks, but fail ZK verification — these still consume the full verification weight. Constructing such proofs requires knowledge of valid on-chain state (real block hashes, unused nullifiers, correct fee parameters), which raises the bar compared to random bytes but does not eliminate the vector.

Longer-term: third-party relayer model.

A relayer network would close the remaining gap while preserving recipient anonymity. In this model, the proof recipient generates their proof offline and publishes it to a relay network. A relayer submits it as a signed transaction, paying standard Substrate fees. On successful verification, the relayer is reimbursed from the bridged amount (or collects a small fee from the output). The recipient never interacts with the chain directly.

Relayers have an economic incentive to only submit proofs they expect to succeed (by verifying locally first, as the CLI already does). Spam is deterred because relayers bear the fee cost for failed proofs. This is the standard approach in privacy-preserving bridges for exactly this reason. The existing codebase suggests a layer-1 aggregation role was considered but is not yet implemented/abandoned.

Fully eliminating the DoS vector requires removing the unsigned extrinsic, which means users who want to preserve anonymity must rely on third-party relayer services. This is a trade-off between spam resistance and self-sovereign submission that should be an explicit design decision.

EQ-QNT-WORMHOLE-F-03: Miner Rewards Silently Burned Due to Incorrect Preimage Handling

The wormhole pallet extracts the block author address from PreRuntime digests incorrectly by attempting to decode the preimage directly as an AccountId, instead of hashing it with Poseidon2 to derive the wormhole address. This causes miner fee distribution to silently fail, burning fees that should be paid to miners. The bug has zero test coverage and would only manifest in production.

Issue

The consensus engine stores a 32-byte preimage in the PreRuntime digest, which must be hashed with Poseidon2 to derive the actual wormhole address. The wormhole pallet incorrectly decodes this preimage directly as an AccountId:

Wormhole pallet (INCORRECT):

pallets/wormhole/src/lib.rs:436-452

```
436 |       // Mint miner's portion of volume fee to block author
437 |       // The remaining portion (fee - miner_fee) is not minted, effectively burned
438 |       if !miner_fee.is_zero() {
439 |         if let Some(author) = frame_system::Pallet::<T>::digest()
440 |           .logs
441 |           .iter()
442 |           .find_map(|item| item.as_pre_runtime())
443 |           .and_then(|(_, data)| {
444 |             <T as frame_system::Config>::AccountId::decode(&mut &data[..]).ok()
445 |           }) {
446 |           <T::Currency as Unbalanced<_>>::increase_balance(
447 |             &author,
448 |             miner_fee,
449 |             frame_support::traits::tokens::Precision::Exact,
450 |           )?;
451 |         }
452 |       }
```

The code has three problems:

1. **Direct decode instead of hashing**: Attempts `AccountId::decode(&mut &data[..])` directly on the preimage bytes.
2. **No engine ID filtering**: Uses `.find_map(|item| item.as_pre_runtime())` which returns ANY PreRuntime digest, not specifically the QPoW digest.
3. **Silent failure**: When extraction fails, the miner fee is burned without emitting an error or event.

Correct Implementation

The mining-rewards pallet implements this correctly:

pallets/mining-rewards/src/lib.rs:187-217

```
187 |     /// Extract miner wormhole address by hashing the preimage from pre-runtime digest
188 |     fn extract_miner_from_digest() -> Option<T::AccountId> {
189 |       // Get the digest from the current block
190 |       let digest = <frame_system::Pallet<T>>::digest();
191 |
192 |       // Look for pre-runtime digest with POW_ENGINE_ID
193 |       for log in digest.logs.iter() {
194 |         if let DigestItem::PreRuntime(engine_id, data) = log {
```

```
          if engine_id == &POW_ENGINE_ID {
            // The data is a 32-byte preimage from the incoming block
            if data.len() == 32 {
              let preimage: [u8; 32] = match data.as_slice().try_into() {
                Ok(arr) => arr,
                Err(_) => continue,
              };

              // Hash the preimage with Poseidon2 to derive the wormhole address
              let wormhole_address_bytes = PoseidonHasher::hash_padded(&preimage);

              // Convert to AccountId
              if let Ok(miner) =
                T::AccountId::decode(&mut &wormhole_address_bytes[..])
              {
                return Some(miner);
              }
            }
          }
        }
      }
      None
    }
```

This approach:

1. Hashes the 32-byte preimage with `PoseidonHasher::hash_padded(&preimage)` to derive the wormhole address
2. Filters by `POW_ENGINE_ID` to ensure the digest comes from the consensus engine
3. Returns `None` explicitly when extraction fails (allowing caller to handle the error)

How PreRuntime digest is populated

When the consensus engine builds a block, it adds the rewards preimage to the block digest:

client/consensus/qpow/src/lib.rs:481-483

```
      let mut inherent_digest = Digest::default();
      let rewards_preimage_bytes = rewards_preimage.to_vec();
      inherent_digest.push(DigestItem::PreRuntime(POW_ENGINE_ID, rewards_preimage_bytes));
```

The `rewards_preimage` is a 32-byte value derived from an `AccountId32` provided via the `--rewards-preimage` CLI argument (or the Treasury account in dev mode):

node/src/command.rs:496-521

```
        let rewards_account = match cli.rewards_preimage {
          Some(address) => {
            let account = address.parse::<AccountId32>().map_err(|_| {
              sc_cli::Error::Input("Invalid rewards preimage format".into())
```

```
500 |           })?;
501 |           log::info!("⛏️ Using address for rewards: {:?}", account);
502 |           account
503 |         },
504 |         None => {
505 |           // Automatically set rewards_preimage to Treasury when --dev is used
506 |           if cli.run.shared_params.is_dev() {
507 |             let treasury_account =
508 |               quantus_runtime::configs::TreasuryPalletId::get()
509 |                 .into_account_truncating();
510 |             log::info!(
511 |               "⛏️ Using treasury address for rewards: {:?}",
512 |               treasury_account
513 |             );
514 |
515 |             treasury_account
516 |           } else {
517 |             // Should never happen
518 |             return Err(sc_cli::Error::Input("No rewards preimage provided".into()));
519 |           }
520 |         },
521 |       };
```

The CLI flag name and variable names are misleading: the argument is described as a “rewards preimage” but is parsed as an `AccountId32` (line 498). The raw account bytes are then used as the preimage, which must be hashed with Poseidon2 to get the wormhole address. The node service converts the `AccountId32` into the 32-byte preimage:

node/src/service.rs:439-449

```
439 | // Start the mining worker (block building task)
440 | // Convert AccountId32 to [u8; 32] for the mining worker (rewards preimage)
441 | let rewards_preimage: [u8; 32] = rewards_address.into();
442 | let (worker_handle, worker_task) = sc_consensus_qpow::start_mining_worker(
443 |   Box::new(pow_block_import),
444 |   client.clone(),
445 |   select_chain,
446 |   proposer,
447 |   sync_service.clone(),
448 |   sync_service.clone(),
449 |   rewards_preimage,
```

This aligns with QIP-5: the preimage must be hashed with Poseidon to derive the wormhole address.

Impact

All miner volume fees are silently burned instead of being distributed to miners due to invalid reward address derivation.

The pallet also assumes only one consensus engine is in operation, so only one PreRuntime digest exists. This is true today, but the pallet should validate it anyway.

Zero Test Coverage

The miner fee extraction code has no test coverage:

- Unit tests (`pallets/wormhole/src/tests.rs`) only test transfer recording
- Benchmark (`pallets/wormhole/src/benchmarking.rs`) uses `RawOrigin::None` which doesn't set up digests
- Mock runtime (`pallets/wormhole/src/mock.rs`) has no digest configuration
- No tests verify miner fee distribution at all

Recommendations

1. **Fix the extraction logic** in `verify_aggregated_proof` (lines 436-452). Since the same logic is needed for both wormhole and mining-rewards pallets, one could extract a common helper:

```
fn extract_author() -> Option<T::AccountId> {
    use qp_poseidon::PoseidonHasher;
    use sp_runtime::traits::Hash as _;

    for log in frame_system::Pallet::<T>::digest().logs.iter() {
        if let DigestItem::PreRuntime(engine_id, data) = log {
            // Filter by POW_ENGINE_ID
            if engine_id == &sp_consensus_pow::POW_ENGINE_ID {
                if data.len() == 32 {
                    let preimage: [u8; 32] = data.as_slice().try_into().ok()?;
                    // Hash preimage with Poseidon to get wormhole address
                    let wormhole_address_bytes = PoseidonHasher::hash_padded(&preimage);
                    return T::AccountId::decode(&mut &wormhole_address_bytes[..]).ok();
                }
            }
        }
    }
    None
}
```

2. **Emit an error or event** when author extraction fails, so silent burns are more easily detected

3. **Add tests**:
    - Test miner fee distribution with valid PreRuntime digest
    - Test behavior when PreRuntime digest is missing
    - Test behavior with incorrect engine ID

- Test behavior with malformed preimage data

4. **Consider refactoring** to share author extraction logic between mining-rewards and wormhole pallets to prevent divergence

5. **Consider adding a separate type for the preimage**. Since preimage is parsed from digest bytes, it would still be possible to interpret them incorrectly. Nevertheless, there are multiple places where account address and account address preimage are mixed (they are the same size), so marking the type explicitly could help spot incorrect usage.

EQ-QNT-WORMHOLE-I-01: Mining Rewards Benchmark Confuses Account Address with Preimage

This is another instance of account address and account preimage being used interchangeably (see EQ-QNT-WORMHOLE-F-03). The benchmark constructs the PreRuntime digest using an encoded AccountId where a raw preimage is expected:

pallets/mining-rewards/src/benchmarking.rs:31-31

```
31 |    let miner_digest_item = DigestItem::PreRuntime(POW_ENGINE_ID, miner.encode());
```

The runtime will Poseidon-hash these bytes to derive the wormhole address, so rewards go to `Poseidon2(miner_bytes)` rather than `miner`. This has no material effect on the benchmark — it only measures the weight of `on_finalize` and does not verify balances — but is misleading.

Recommendation
Use a raw 32-byte preimage instead of `miner.encode()` for clarity.

EQ-QNT-WORMHOLE-F-04: Non-Native Asset Transfers Unsupported Despite Partial Implementation

The codebase is of two minds about non-native asset transfers. The pallet runtime has full branching logic to mint non-native assets via `FungiblesMutate`, including event variants and transfer proof recording. At the same time, the proof aggregation pipeline – the only path for submitting wormhole proofs on-chain – explicitly forbids non-zero `asset_id` at multiple layers.

QIP-0005 explicitly intends non-native asset support. The spec lists `asset_id` as a public input:

qip-0005.md:31-31

- `asset_id`: The identifier for the asset being transferred (0 for native token).

And the rationale states:

qip-0005.md:186-186

> The asset_id field enables support for multiple token types beyond the native token, allowing the wormhole system to handle various assets.

Pallet: non-native assets fully wired

The `verify_aggregated_proof` extrinsic extracts `asset_id` from the aggregated public inputs and dispatches accordingly – native tokens through `Currency::increase_balance`, non-native through `Assets::mint_into`:

pallets/wormhole/src/lib.rs:409-425

```
409 |         // Handle native (asset_id = 0) or asset transfers
410 |         if asset_id == AssetIdOf::<T>::default() {
411 |           // Native token transfer - mint tokens to the exit account
412 |           <T::Currency as Unbalanced<_>>::increase_balance(
413 |             exit_account,
414 |             *exit_balance,
415 |             frame_support::traits::tokens::Precision::Exact,
416 |           )?;
417 |         } else {
418 |           // Asset transfer
419 |           let asset_balance: AssetBalanceOf<T> = (*exit_balance).into();
420 |           <T::Assets as FungiblesMutate<_>>::mint_into(
421 |             asset_id.clone(),
422 |             exit_account,
423 |             asset_balance,
424 |           )?;
425 |         }
```

The `record_transfer` helper similarly branches on `asset_id`, emitting either `NativeTransferred` or `AssetTransferred`:

pallets/wormhole/src/lib.rs:503-518

```
503 |       if asset_id == AssetIdOf::<T>::default() {
504 |         Self::deposit_event(Event::<T>::NativeTransferred {
505 |           from: from.into(),
506 |           to: to.into(),
507 |           amount,
508 |           transfer_count: current_count,
509 |         });
510 |       } else {
511 |         Self::deposit_event(Event::<T>::AssetTransferred {
512 |           from: from.into(),
513 |           to: to.into(),
514 |           asset_id,
```

```
            amount: amount.into(),
            transfer_count: current_count,
          });
        }
```

Dummy proofs: `asset_id` hardcoded to 0

Dummy proofs used for batch padding hardcode `asset_id = 0`:

wormhole/aggregator/src/dummy_proof.rs:150-162

```
    Ok(CircuitInputs {
        public: PublicCircuitInputs {
            asset_id: 0u32,
            output_amount_1: DEFAULT_OUTPUT_AMOUNT, // Dummy proofs output 0
            output_amount_2: 0u32,                  // No second output for dummies
            volume_fee_bps: DEFAULT_VOLUME_FEE_BPS,
            nullifier,
            exit_account_1: exit_account,
            exit_account_2: BytesDigest::default(), // No second exit account
            // Sentinel: block_hash = 0 triggers validation bypass
            block_hash: BytesDigest::try_from(DUMMY_BLOCK_HASH)?,
            block_number: DEFAULT_BLOCK_NUMBER,
        },
```

Aggregator: explicit pre-aggregation rejection

Before padding with dummies, the aggregator checks that real proofs also carry `asset_id = 0` and bails otherwise:

wormhole/aggregator/src/aggregator.rs:204-231

```
    /// Aggregates `N` number of leaf proofs into an [`AggregatedProof`].
    ///
    /// # Note
    /// Pre-generated dummy proofs use `asset_id = 0` (native token). All real proofs
    /// must also use `asset_id = 0` for the aggregation to succeed, since the circuit
    /// enforces all proofs have the same asset_id.
    pub fn aggregate(&mut self) -> anyhow::Result<AggregatedProof> {
        let Some(mut proofs) = self.proofs_buffer.take() else {
            bail!("there are no proofs to aggregate")
        };

        // Pad with dummy proofs if needed
        let num_dummies_needed = self.config.num_leaf_proofs.saturating_sub(proofs.len());
        if num_dummies_needed > 0 {
            // Verify asset_id matches dummy proofs (asset_id = 0)
            // We cannot modify public inputs post-proof as that invalidates the proof.
            if let Some(first_proof) = proofs.first() {
                let real_asset_id: u32 = first_proof.public_inputs[0]
                    .to_canonical_u64()
```

```
                    .try_into()
                    .context("asset_id in first proof exceeds u32 range")?;
                if real_asset_id != 0 {
                    bail!(
                        "Real proofs have asset_id={}, but dummy proofs use asset_id=0. \
                         All proofs must have the same asset_id for aggregation.",
                        real_asset_id
                    );
                }
```

Circuit: unconditional `asset_id` equality constraint

The aggregation wrapper circuit enforces that every proof in the batch has the same `asset_id` via a hard `builder.connect` – unlike `block_hash`, which has a dummy sentinel bypass:

wormhole/aggregator/src/circuits/tree.rs:274-279

```
        // Also enforce asset_id and volume_fee_bps consistency
        let asset_i = limb1_at_offset::<LEAF_PI_LEN, ASSET_ID_START>(child_pi_targets, i);
        builder.connect(asset_i, asset_ref);
        let volume_fee_bps_i =
            limb1_at_offset::<LEAF_PI_LEN, VOLUME_FEE_BPS_START>(child_pi_targets, i);
        builder.connect(volume_fee_bps_i, volume_fee_bps_ref);
```

What would need to change

Supporting non-native asset aggregation requires changes at every layer:

0. **Dummy proofs**: parameterize `build_dummy_circuit_inputs()` to accept an `asset_id`, or generate per-asset dummy proof templates at aggregator startup.
1. Alternatively, **Aggregation circuit**: add a dummy bypass for `asset_id` (i.e., `is_dummy OR asset_id == asset_ref`).
2. **Aggregator**: remove the `asset_id != 0` bail and group proofs by `asset_id` before batching.
3. **Fee handling**: `verify_aggregated_proof` uses `Currency::burn` unconditionally. Non-native assets would need asset-specific fee accounting.

Recommendations

- If non-native asset aggregation is planned, implement the changes above.
- If it is not planned, remove the dead `asset_id` branching from `verify_aggregated_proof` and `record_transfer` to avoid implying support that does not exist, and document the native-only limitation.

EQ-QNT-WORMHOLE-F-05: Transfer Variants Missing from Wormhole Proof Recorder

The `WormholeProofRecorderExtension` transaction extension records wormhole transfer proofs for on-chain transfers. It pattern-matches on specific call variants to extract transfer details:

runtime/src/transaction_extensions.rs:129-167

```
129 |     match call {
130 |       // Native balance transfers
131 |       RuntimeCall::Balances(pallet_balances::Call::transfer_keep_alive { dest, value }) => {
132 |         let to = Self::lookup(dest)?;
133 |         Ok(vec![TransferDetails { from: who.clone(), to, amount: *value, asset_id: 0 }])
134 |       },
135 |       RuntimeCall::Balances(pallet_balances::Call::transfer_allow_death { dest, value }) => {
136 |         let to = Self::lookup(dest)?;
137 |         Ok(vec![TransferDetails { from: who.clone(), to, amount: *value, asset_id: 0 }])
138 |       },
139 |
140 |       // Asset transfers
141 |       RuntimeCall::Assets(pallet_assets::Call::transfer { id, target, amount }) => {
142 |         let to = Self::lookup(target)?;
143 |         Ok(vec![TransferDetails { asset_id: id.0, from: who.clone(), to, amount: *amount }])
144 |       },
145 |       RuntimeCall::Assets(pallet_assets::Call::transfer_keep_alive {
146 |         id,
147 |         target,
148 |         amount,
149 |       }) => {
150 |         let to = Self::lookup(target)?;
151 |         Ok(vec![TransferDetails { asset_id: id.0, from: who.clone(), to, amount: *amount }])
152 |       },
153 |
154 |       // Batch calls -- recurse into inner calls
155 |       RuntimeCall::Utility(pallet_utility::Call::batch { calls }) |
156 |       RuntimeCall::Utility(pallet_utility::Call::batch_all { calls }) |
157 |       RuntimeCall::Utility(pallet_utility::Call::force_batch { calls }) => {
158 |         let mut all = Vec::new();
159 |         for inner_call in calls {
160 |           all.extend(Self::extract_all_transfers(who, inner_call)?);
161 |         }
162 |         Ok(all)
163 |       },
164 |
165 |       // Everything else (transfer_all, system calls, etc.)
166 |       _ => Ok(vec![]),
167 |     }
```

Only four variants are handled: `Balances::transfer_keep_alive`, `Balances::transfer_allow_death`, `Assets::transfer`, and `Assets::transfer_keep_alive`. All other transfer calls fall through to the catch-all branch and produce no proof record.

Notable unhandled variants include:

- `Balances::transfer_all` — transfers the sender's entire free balance. The amount is not known until execution, so it cannot be extracted in `prepare`.
- `Assets::transfer_approved` — a delegated transfer where a pre-approved third party spends on behalf of the asset owner. The `from` account is not the transaction signer, which the current extraction logic assumes.

Since each wormhole withdrawal must reference a specific inbound transfer proof, funds moved via unhandled variants have no corresponding proof and cannot be withdrawn through the wormhole. Those funds are effectively lost to the bridge.

The test suite only exercises the four supported variants. There are no tests for unsupported transfer types to assert that the omission is intentional.

Recommendation
Document the supported transfer variants. For `transfer_all`, consider recording the proof in `post_dispatch` where the actual transferred amount is available from the dispatch result.

EQ-QNT-DOC-F-01: Leftover Amount Redemption Issue in QIP-5

QIP-5 claims users can withdraw partial amounts and later redeem remaining balances:

qip-0005.md:153-155

> 1. **Quantization**: The output amount is quantized with 0.01 units of precision, reducing the number of significant bits exposed. This means many different input amounts map to the same quantized output, creating ambiguity for attackers trying to match transactions by amount.
> 2. **User-Controlled Output Reduction**: Users can specify an output amount lower than their maximum entitlement (input minus fees). By voluntarily leaving some funds behind or withdrawing less than the full amount, users can break the direct correlation between input and output amounts. The remaining balance can be withdrawn later to a different address.

The following sentence is incorrect:

"**The remaining balance can be withdrawn later to a different address.**"

The nullifier mechanism prevents multiple withdrawals from the same deposit. It is defined as:

```
nullifier = H(H("~nullif~" || secret || transfer_count))
```

Key constraints:

- `secret`: Fixed per unspendable address
- `transfer_count`: Per-recipient counter from the specific `TransferProof` entry
- Once used, nullifier is permanently marked in `UsedNullifiers` storage

Example: User deposits 100 tokens (transfer_count = 0):

1. First withdrawal: `output_amount = 60`, generates `nullifier = H(H("~nullif~" || secret || 0))`
2. Nullifier marked as used
3. Second withdrawal attempt: Same secret + same transfer_count = identical nullifier results in `NullifierAlreadyUsed` error

The remaining 40 tokens are permanently inaccessible.

Code References

- Nullifier derivation:

wormhole/circuit/src/nullifier.rs:66-86

```
66 |     pub fn from_preimage(secret: BytesDigest, transfer_count: u64) -> Self {
67 |         let mut preimage = Vec::new();
68 |
69 |         let salt = injective_string_to_felt(NULLIFIER_SALT);
70 |         let secret = digest_bytes_to_felts(secret);
71 |         let transfer_count = u64_to_felts(transfer_count);
72 |
73 |         preimage.extend(salt);
74 |         preimage.extend(secret);
75 |         preimage.extend(transfer_count);
76 |
77 |         let inner_hash = Poseidon2Hash::hash_no_pad(&preimage).elements;
78 |         let outer_hash = Poseidon2Hash::hash_no_pad(&inner_hash).elements;
79 |         let hash = Digest::from(outer_hash);
80 |
81 |         Self {
82 |             hash,
83 |             secret,
84 |             transfer_count,
85 |         }
86 |     }
```

- On-chain check:

pallets/wormhole/src/lib.rs:266-276

```
      for nullifier in &aggregated_inputs.nullifiers {
        let nullifier_bytes: [u8; 32] = (*nullifier)
          .as_ref()
          .try_into()
          .map_err(|_| Error::<T>::InvalidAggregatedPublicInputs)?;
        ensure!(
          !UsedNullifiers::<T>::contains_key(nullifier_bytes),
          Error::<T>::NullifierAlreadyUsed
        );
        UsedNullifiers::<T>::insert(nullifier_bytes, true);
      }
```

- Transfer count:

pallets/wormhole/src/lib.rs:499-501

```
        (),
      );
      TransferCount::<T>::insert(&to, current_count.saturating_add(T::TransferCount::one()));
```

Impact

The two-output solution introduced in the current implementation largely mitigates the practical impact: users can split a deposit into a withdrawal output and a change output in a single proof, avoiding stranded funds. However, the QIP-5 specification still contains the incorrect claim that remaining balances can be withdrawn in a separate, later transaction, which is impossible due to the nullifier mechanism.

Recommendations

Correct QIP-5 specification to remove the claim about redeeming leftover amounts via separate withdrawals, and document the two-output mechanism as the intended way to handle partial withdrawals.

EQ-QNT-WORMHOLE-R-01: Unused Error Variants in Wormhole Pallet

The wormhole pallet's `Error` enum contains seven variants that are never used anywhere in the codebase:

pallets/wormhole/src/lib.rs:208-228

```
#[pallet::error]
pub enum Error<T> {
  InvalidProof,
  ProofDeserializationFailed,
  VerificationFailed,
```

```
213 |     InvalidPublicInputs,
214 |     NullifierAlreadyUsed,
215 |     VerifierNotAvailable,
216 |     InvalidStorageRoot,
217 |     StorageRootMismatch,
218 |     BlockNotFound,
219 |     InvalidBlockNumber,
220 |     AggregatedVerifierNotAvailable,
221 |     AggregatedProofDeserializationFailed,
222 |     AggregatedVerificationFailed,
223 |     InvalidAggregatedPublicInputs,
224 |     /// The volume fee rate in the proof doesn't match the configured rate
225 |     InvalidVolumeFeeRate,
226 |     /// Transfer amount is below the minimum required
227 |     TransferAmountBelowMinimum,
228 |   }
```

The unused variants are:

- `InvalidProof`
- `ProofDeserializationFailed`
- `VerificationFailed`
- `VerifierNotAvailable`
- `InvalidStorageRoot`
- `StorageRootMismatch`
- `InvalidBlockNumber`

These appear to be remnants of the removed `verify_wormhole_proof` (single-proof) extrinsic. The aggregated proof path uses its own error variants (`AggregatedVerifierNotAvailable`, `AggregatedProofDeserializationFailed`, etc.), leaving the original set dead.

`InvalidBlockNumber` specifically was made redundant when the explicit `block_number <= current_block` check was removed in favor of the `block_hash != default_hash` check alone (see lines 284-289), which correctly covers future, pruned, and invalid blocks in one step.

Recommendations

Remove the seven unused error variants.

EQ-QNT-WORMHOLE-R-02: `volume_fee_bps` Lacks a Single Source of Truth

The aggregation pipeline spans four components — the circuit builder, the aggregator, the CLI, and the on-chain verifier. Parameters that must match across all of them benefit from a single source of truth. `num_leaf_proofs` has one (`config.json`); `volume_fee_bps` does not.

`num_leaf_proofs`: single source of truth

The batch size is set once at circuit-build time and persisted in `config.json`:

wormhole/aggregator/src/config.rs:56-65

```rust
/// Configuration stored alongside circuit binaries (config.json).
/// This struct is used by both circuit-builder (to save config) and
/// aggregator (to load config when aggregating proofs).
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CircuitBinsConfig {
    pub num_leaf_proofs: usize,
    /// SHA256 hashes of the binary files for integrity verification
    #[serde(default, skip_serializing_if = "Option::is_none")]
    pub hashes: Option<BinaryHashes>,
}
```

The CLI and aggregator read it from this file. The pallet derives it from the public inputs length.

`volume_fee_bps`: five independent copies of the same magic number

The fee rate must match across prover, aggregator, and verifier. Unlike `num_leaf_proofs`, it has no config file entry and no shared constant. The value `10` is hardcoded independently in five places. Three in production code:

Runtime: *runtime/src/configs/mod.rs:691-691*

```rust
 pub const VolumeFeeRateBps: u32 = 10;
```

CLI: *src/cli/wormhole.rs:53-53*

```rust
pub const VOLUME_FEE_BPS: u32 = 10;
```

Dummy proof builder: *wormhole/aggregator/src/dummy_proof.rs:61-61*

```rust
const DEFAULT_VOLUME_FEE_BPS: u32 = 10;
```

Two more in tests and examples:

Test helpers: *wormhole/tests/test-helpers/src/lib.rs:32-32*

```rust
pub const DEFAULT_VOLUME_FEE_BPS: u32 = 10; // 0.1% = 10 basis points
```

End-to-end example: *wormhole/example/src/main.rs:45-45*

```
const VOLUME_FEE_BPS: u32 = 10; // 0.1% = 10 basis points
```

In practice, a mismatch is caught early: `volume_fee_bps` is a public input, and the pallet rejects proofs with a mismatched rate via `InvalidVolumeFeeRate`. The aggregator also pre-validates that all leaves agree on the fee rate before proving. So this is unlikely to cause a prolonged outage — but it is still five places to update instead of one.

Recommendation

Add `volume_fee_bps` to `config.json` alongside `num_leaf_proofs`, and have all consumers read it from there.

EQ-QNT-WORMHOLE-R-04: Dummy Proof Detection Uses Different Conditions Across the Stack

The documentation in `dummy_proof.rs` defines the dummy sentinel as all three conditions together:

wormhole/aggregator/src/dummy_proof.rs:8-12

```
//! # Sentinel Values
//!
//! - `block_hash = [0u8; 32]` AND `output_amount_1 = 0` AND `output_amount_2 = 0`:
//!   Triggers bypass of all validation. Both conditions must be met to prevent
//!   an attacker from slipping funds through with a zero block hash.
```

Five sites detect dummy proofs, each using a different subset of conditions:

Site	Conditions checked
Leaf circuit	`block_hash == 0` AND `output_1 == 0` AND `output_2 == 0`
Storage proof witness	`block_hash == 0` AND `output_1 == 0` AND `output_2 == 0`
Layer-0 aggregation	`block_hash == 0` only
Layer-1 aggregation	`block_hash == 0` only
Pallet runtime	`exit_account == 0` OR `summed_output_amount == 0`

Leaf circuit and storage proof witness

The leaf circuit computes `is_dummy` as the AND of three zero-checks, matching the documented sentinel:

wormhole/circuit/src/circuit.rs:142-167

```
142 |         // Dummy proof detection: requires BOTH block_hash == 0 AND output_amounts == 0.
143 |         // This prevents an attacker from slipping funds through with a zero block hash
144 |         // but positive output amounts.
145 |         let zero = builder.zero();
146 |         let one = builder.one();
147 |
148 |         // Check if all four limbs of block_hash are zero
149 |         let bh = &targets.block_header.block_hash.elements;
150 |         let bh0_is_zero = builder.is_equal(bh[0], zero);
151 |         let bh1_is_zero = builder.is_equal(bh[1], zero);
152 |         let bh2_is_zero = builder.is_equal(bh[2], zero);
153 |         let bh3_is_zero = builder.is_equal(bh[3], zero);
154 |
155 |         let bh01_zero = builder.and(bh0_is_zero, bh1_is_zero);
156 |         let bh23_zero = builder.and(bh2_is_zero, bh3_is_zero);
157 |         let block_hash_is_zero = builder.and(bh01_zero, bh23_zero);
158 |
159 |         // Check if both output amounts are individually zero
160 |         let leaf = &targets.storage_proof.leaf_inputs;
161 |         let output_1_is_zero = builder.is_equal(leaf.output_amount_1, zero);
162 |         let output_2_is_zero = builder.is_equal(leaf.output_amount_2, zero);
163 |         let both_outputs_zero = builder.and(output_1_is_zero, output_2_is_zero);
164 |
165 |         // is_dummy = block_hash_is_zero AND both_outputs_zero
166 |         let is_dummy = builder.and(block_hash_is_zero, both_outputs_zero);
167 |         let is_not_dummy = builder.sub(one, is_dummy.target);
```

This flag gates nullifier validation, block hash validation, state root matching, and merkle path verification. The storage proof witness generation uses the same triple condition:

wormhole/circuit/src/storage_proof/mod.rs:113-116

```
113 |         // If the block hash is zero and output amounts are zero, we treat this as a dummy
proof and set is_not_dummy to false.
114 |         let is_not_dummy = !(inputs.public.block_hash.as_ref() == [0u8; 32]
115 |             && inputs.public.output_amount_1 == 0
116 |             && inputs.public.output_amount_2 == 0);
```

Aggregation wrapper

The wrapper detects dummies by checking `block_hash` alone:

wormhole/aggregator/src/circuits/tree.rs:113-119

```
113 | impl AggregationWrapper for WormholeAggregationWrapper {
114 |     fn is_dummy(&self, proof: &ProofWithPublicInputs<F, C, D>) -> bool {
115 |         // Wormhole dummy proofs have block_hash == [0,0,0,0]
116 |         proof.public_inputs[BLOCK_HASH_START..BLOCK_HASH_START + 4]
117 |             .iter()
118 |             .all(|f| f.is_zero())
119 |     }
```

The in-circuit dummy flag uses the same single condition:

wormhole/aggregator/src/circuits/tree.rs:258-263

```
258 |     for i in 0..n_leaf {
259 |         let block_i = limbs4_at_offset::<LEAF_PI_LEN, BLOCK_HASH_START>(child_pi_targets, i);
260 |
261 |         // is_dummy_i = (block_i == [0,0,0,0])
262 |         let is_dummy_i = bytes_digest_eq(&mut builder, block_i, dummy_sentinel);
263 |         is_dummy_flags.push(is_dummy_i);
```

This flag is used to skip the same-block constraint and to replace dummy nullifiers with random values.

Layer-1 aggregation wrapper

Layer-1 repeats the block-hash-only check over layer-0 output headers:

wormhole/aggregator/src/circuits/layer1.rs:88-95

```
88 |     fn is_dummy(&self, proof: &ProofWithPublicInputs<F, C, D>) -> bool {
89 |         // A dummy layer-0 aggregated proof has block_hash == [0,0,0,0] in its header.
90 |         // This propagates from the layer-0 wrapper: if all leaf proofs are dummies,
91 |         // the layer-0 output's block_hash is [0,0,0,0].
92 |         proof.public_inputs[l0::BLOCK_HASH_OFFSET..l0::BLOCK_HASH_OFFSET + 4]
93 |             .iter()
94 |             .all(|f| f.is_zero())
95 |     }
```

wormhole/aggregator/src/circuits/layer1.rs:156-163

```
156 |             // Enforce same block (or dummy)
157 |             let block_i: [Target; 4] =
158 |                 core::array::from_fn(|j| child_pis[base + l0::BLOCK_HASH_OFFSET + j]);
159 |             let is_dummy = bytes_digest_eq(&mut builder, block_i, dummy_sentinel);
160 |             let matches_ref = bytes_digest_eq(&mut builder, block_i, block_ref);
161 |             let ok = builder.or(is_dummy, matches_ref);
162 |             builder.connect(ok.target, one);
163 |         }
```

Note that layer-1 aggregation is currently unused.

Pallet runtime

On-chain, the pallet skips slots where `exit_account == 0` OR `summed_output_amount == 0`:

pallets/wormhole/src/lib.rs:317-328

```
317 |         // Skip dummy account slots (exit_account == 0 with zero amount)
318 |         // Dummy proofs from aggregation padding have all-zero exit accounts
319 |         // Also skip deduplicated slots (the circuit zeros out duplicate exit accounts)
320 |         let exit_account_bytes: [u8; 32] =
321 |           (*account_data.exit_account).as_ref().try_into().map_err(|e| {
322 |             log::error!("Failed to convert exit_account at idx {}: {:?}", idx, e);
323 |             Error::<T>::InvalidAggregatedPublicInputs
324 |           })?;
325 |
326 |         if exit_account_bytes == [0u8; 32] || account_data.summed_output_amount == 0 {
327 |           continue;
328 |         }
```

This also catches deduplicated slots (where the circuit zeros out duplicate exit accounts). Note the inline comment on line 317 says "exit_account == 0 with zero amount", implying AND, but the actual condition on line 326 uses OR.

Current behavior

None of these differences cause incorrect behavior. A proof with `block_hash == 0` and non-zero outputs is rejected by the leaf circuit — `is_not_dummy` evaluates to `1`, enforcing `block_hash == H(header_contents)`, which requires a Poseidon2 preimage hashing to zero. The only proofs that pass verification with `block_hash == 0` are those with both outputs zero. The pallet's OR condition is similarly safe because the aggregation circuit guarantees dummy slots arrive with both zero exit account and zero amount.

If the sentinel definition changes, having five separate check sites makes it easy to update some and miss others.

Recommendation

Where a site intentionally uses a narrower or broader condition (e.g., the aggregation wrappers relying on `block_hash` alone being sufficient post-verification, or the pallet also catching deduplicated slots), add an explicit comment explaining why it differs from the canonical sentinel. Alternatively, extract a single shared predicate that all sites use, so a change to the definition only needs to happen once.

EQ-QNT-WORMHOLE-R-05: Storage Proof Preprocessing Operates on Hex Strings Instead of Binary Data

The `prepare_proof_for_circuit` function in `qp-zk-circuits/common/src/storage_proof.rs` reconstructs the Merkle path from root to leaf by converting every trie node to a hex-encoded string and performing substring searches on these strings. This applies to root identification, parent–child path walking, index computation, and leaf hash verification.

common/src/storage_proof.rs:85-94

```
    let mut node_map: alloc::collections::BTreeMap<String, (usize, Vec<u8>, String)> =
        alloc::collections::BTreeMap::new();

    for (idx, node) in proof.iter().enumerate() {
        let node_bytes = node.as_ref();
        let hash = hash_node_with_poseidon_padded(node_bytes);
        let hash_hex = hex::encode(hash);
        let node_hex = hex::encode(node_bytes);
        node_map.insert(hash_hex.clone(), (idx, node_bytes.to_vec(), node_hex));
    }
```

common/src/storage_proof.rs:118-140

```
    loop {
        // Try to find which node's hash appears in the current node
        // Child hashes are prefixed with their length (32 bytes = 0x2000000000000000 in little-endian)
        let mut found_child = None;
        for (child_hash, (_, child_bytes, _)) in &node_map {
            let hash_with_prefix = format!("{}{}", HASH_LENGTH_PREFIX, child_hash);
            if current_node_hex.contains(&hash_with_prefix) {
                // Make sure we haven't already added this node
                if !ordered_nodes.iter().any(|n| n == child_bytes) {
                    found_child = Some(child_bytes.clone());
                    break;
                }
            }
        }

        if let Some(child_bytes) = found_child {
            ordered_nodes.push(child_bytes.clone());
            current_node_hex = hex::encode(ordered_nodes.last().unwrap());
        } else {
            // No more children found - we've reached the end of the proof path
            break;
        }
    }
```

common/src/storage_proof.rs:146-156

```
    for i in 0..ordered_nodes.len() - 1 {
        let current_hex = hex::encode(&ordered_nodes[i]);
        let next_node = &ordered_nodes[i + 1];
        let next_hash = hex::encode(hash_node_with_poseidon_padded(next_node));

        if let Some(hex_idx) = current_hex.find(&next_hash) {
            indices.push(hex_idx);
        } else {
            bail!("Could not find child hash in ordered node {}", i);
        }
    }
```

Every byte becomes two ASCII characters, doubling memory use and requiring allocation of owned `Strings` for every node and hash. While `str::find` is better optimized than a naive byte scan, the cost of hex encoding and the associated allocations introduce unnecessary overhead, potentialy mitigating any gains there.

The `check_leaf` function, while operating on binary data directly, uses a naive byte-by-byte scan to locate the 24-byte suffix.

It also returns `(bool, usize)` with an implicit sentinel of `(false, 0)`:

common/src/storage_proof.rs:47-61

```
47 | pub fn check_leaf(leaf_hash: &[u8; 32], leaf_node: &[u8]) -> (bool, usize) {
48 |     let hash_suffix = &leaf_hash[8..32];
49 |     let mut last_idx = 0usize;
50 |     let mut found = false;
51 |
52 |     for i in 0..=leaf_node.len().saturating_sub(hash_suffix.len()) {
53 |         if &leaf_node[i..i + hash_suffix.len()] == hash_suffix {
54 |             last_idx = i;
55 |             found = true;
56 |             break;
57 |         }
58 |     }
59 |
60 |     (found, (last_idx * 2).saturating_sub(16))
61 | }
```

The `saturating_sub(16)` on the returned index silently clamps to zero if the suffix is found within the first 8 bytes of the node—masking what would be malformed input rather than reporting it.

Recommendations

- Replace hex-string operations with searching directly on binary data with `memchr::memmem::find`.
- Replace naive byte search in `check_leaf` with the same.
- Change `check_leaf` to return `Option<usize>` instead of `(bool, usize)`.
- Replace `saturating_sub(16)` with checked arithmetic that returns an error on underflow, so malformed input is caught rather than silently clamped.

EQ-QNT-WORMHOLE-R-07: `WormholeProver::default()` Silently Falls Back to Fresh Circuit Build on Any File Error

The `Default` implementation for `WormholeProver` attempts to load serialized circuit data from a hardcoded relative path, then silently builds the circuit from scratch if that fails for any reason:

wormhole/prover/src/lib.rs:98-118

```
 98 | impl Default for WormholeProver {
 99 |     fn default() -> Self {
100 |         Self::new_from_files(
101 |             Path::new("generated-bins/prover.bin"),
102 |             Path::new("generated-bins/common.bin"),
103 |         )
104 |         .unwrap_or_else(|_| {
105 |             let wormhole_circuit = WormholeCircuit::default();
106 |             let partial_witness = PartialWitness::new();
107 |
108 |             let targets = Some(wormhole_circuit.targets());
109 |             let circuit_data = wormhole_circuit.build_prover();
110 |
111 |             Self {
112 |                 circuit_data,
113 |                 partial_witness,
114 |                 targets,
115 |             }
116 |         })
117 |     }
118 | }
```

This has two problems:

1. The paths `generated-bins/prover.bin` and `generated-bins/common.bin` are relative to the current working directory. Depending on where the binary is invoked from, they may resolve to entirely different files, or to no files at all. There is no validation that the loaded data matches the expected circuit.

2. The `unwrap_or_else` fallback treats all errors identically: missing files, permission errors, and deserialization failures (e.g. loading bins from an incompatible circuit version) all silently fall through to building a fresh circuit. A caller who intends to load pre-built data has no way to know they got a freshly built circuit instead. If the loaded data is corrupt or from a mismatched version, the deserialization error is swallowed, and the prover silently operates with different circuit parameters than the caller expects.

Recommendations

Remove the `Default` implementation. The two operations it conflates—loading from files and building from scratch—are distinct intents that should require explicit constructor calls.

`new_from_files` already exists and returns a `Result`, letting the caller handle errors. A separate `build_fresh` (or similar) constructor would make the fallback path explicit rather than automatic.

EQ-QNT-WORMHOLE-R-08: Infallible `record_transfer` Returns `Result`

The `record_transfer` function returns `DispatchResult`, but its body consists entirely of infallible operations — two storage writes, a saturating addition, and an event deposit:

pallets/wormhole/src/lib.rs:490-521

```
490 |     pub fn record_transfer(
491 |       asset_id: AssetIdOf<T>,
492 |       from: <T as Config>::WormholeAccountId,
493 |       to: <T as Config>::WormholeAccountId,
494 |       amount: BalanceOf<T>,
495 |     ) -> DispatchResult {
496 |       let current_count = TransferCount::<T>::get(&to);
497 |       TransferProof::<T>::insert(
498 |         (asset_id.clone(), current_count, from.clone(), to.clone(), amount),
499 |         (),
500 |       );
501 |       TransferCount::<T>::insert(&to, current_count.saturating_add(T::TransferCount::one()));
502 |
503 |       if asset_id == AssetIdOf::<T>::default() {
504 |         Self::deposit_event(Event::<T>::NativeTransferred {
505 |           from: from.into(),
506 |           to: to.into(),
507 |           amount,
508 |           transfer_count: current_count,
509 |         });
510 |       } else {
511 |         Self::deposit_event(Event::<T>::AssetTransferred {
512 |           from: from.into(),
513 |           to: to.into(),
514 |           asset_id,
515 |           amount: amount.into(),
516 |           transfer_count: current_count,
517 |         });
518 |       }
519 |
520 |       Ok(())
521 |     }
```

The unnecessary `Result` propagates through multiple layers of the codebase:

- The `TransferProofRecorder` trait is forced to define an associated `Error` type and return `Result`:

primitives/wormhole/src/lib.rs:8-21

```
 8 | pub trait TransferProofRecorder<AccountId, AssetId, Balance> {
 9 |  /// Error type for proof recording failures
10 |  type Error;
```

```rust

/// Record a transfer proof for native or asset tokens
/// - `None` for native tokens (asset_id = 0)
/// - `Some(asset_id)` for specific assets
fn record_transfer_proof(
  asset_id: Option<AssetId>,
  from: AccountId,
  to: AccountId,
  amount: Balance,
) -> Result<(), Self::Error>;
}
```

- The trait implementation sets `type Error = DispatchError` and delegates to `record_transfer`:

pallets/wormhole/src/lib.rs:532-542

```rust
    type Error = DispatchError;

    fn record_transfer_proof(
      asset_id: Option<AssetIdOf<T>>,
      from: <T as Config>::WormholeAccountId,
      to: <T as Config>::WormholeAccountId,
      amount: BalanceOf<T>,
    ) -> Result<(), Self::Error> {
      let asset_id_value = asset_id.unwrap_or_default();
      Self::record_transfer(asset_id_value, from, to, amount)
    }
```

- The `mining-rewards` mock must define a `MockProofRecorder` struct with `type Error = ()` that returns `Ok(())`:

pallets/mining-rewards/src/mock.rs:96-110

```rust
pub struct MockProofRecorder;
impl qp_wormhole::TransferProofRecorder<sp_core::crypto::AccountId32, u32, u128>
 for MockProofRecorder
{
 type Error = ();

 fn record_transfer_proof(
   _asset_id: Option<u32>,
   _from: sp_core::crypto::AccountId32,
   _to: sp_core::crypto::AccountId32,
   _amount: u128,
 ) -> Result<(), Self::Error> {
   Ok(())
 }
}
```

- The `mining-rewards` pallet discards the result with `let _ =`, since the call happens in `on_finalize` where errors cannot be propagated:

pallets/mining-rewards/src/lib.rs:240-245

```
240 |         let _ = T::ProofRecorder::record_transfer_proof(
241 |           None,
242 |           mint_account.clone(),
243 |           miner.clone(),
244 |           reward,
245 |         );
```

- The transaction extension defines a wrapper function solely to `.map_err()` the unreachable error into `InvalidTransaction::Custom(100)`:

runtime/src/transaction_extensions.rs:189-199

```
189 |  fn record_proof(details: TransferDetails) -> Result<(), TransactionValidityError> {
190 |    let asset_id = if details.asset_id == 0 { None } else { Some(details.asset_id) };
191 |
192 |    <Wormhole as TransferProofRecorder<AccountId, AssetId, Balance>>::record_transfer_proof(
193 |      asset_id,
194 |      details.from,
195 |      details.to,
196 |      details.amount,
197 |    )
198 |    .map_err(|_| TransactionValidityError::Invalid(InvalidTransaction::Custom(100)))
199 |  }
```

- `post_dispatch` uses `?` on that wrapper, making it appear that proof recording could fail and revert the transaction:

runtime/src/transaction_extensions.rs:252-267

```
252 |  fn post_dispatch(
253 |    pre: Self::Pre,
254 |    _info: &DispatchInfoOf<RuntimeCall>,
255 |    _post_info: &mut PostDispatchInfoOf<RuntimeCall>,
256 |    _len: usize,
257 |    result: &DispatchResult,
258 |  ) -> Result<(), TransactionValidityError> {
259 |    // Only record proofs if the transaction succeeded
260 |    if result.is_ok() {
261 |      for details in pre {
262 |        Self::record_proof(details)?;
263 |      }
264 |    }
265 |
266 |    Ok(())
267 |  }
```

- `verify_aggregated_proof` uses `?` on `record_transfer` inside the exit processing loop, suggesting the extrinsic could revert at that point:

pallets/wormhole/src/lib.rs:427-434

```
427 |         // Record transfer proof for the minted tokens
428 |         Self::record_transfer(
429 |           asset_id.clone(),
430 |           mint_account.clone().into(),
431 |           exit_account.clone().into(),
432 |           *exit_balance,
433 |         )?;
434 |       }
```

If `record_transfer` returned `()`, the trait would not need an `Error` type, the mock could be trivial, the `let _ =` in `mining-rewards` would disappear, the `record_proof` wrapper and its error mapping would be unnecessary, and `post_dispatch` would be a plain loop with no error handling.

Recommendation

Either return `()` from `record_transfer` or document why `DispatchResult` is retained for future use.

EQ-QNT-WORMHOLE-R-06: Inconsistent `UnspendableAccount` Conversion Patterns in Wormhole Example

The wormhole example converts an `UnspendableAccount` to a `SubxtAccountId` in three places, each using a different approach. One call site uses a local `get_unspendable_account` helper:

wormhole/example/src/main.rs:386-401

```
386 | fn get_unspendable_account(
387 |     secret_bytes: [u8; 32],
388 | ) -> anyhow::Result<(BytesDigest, SubxtAccountId, Digest)> {
389 |     let secret: BytesDigest = secret_bytes.try_into()?;
390 |     let unspendable_account =
391 |
wormhole_circuit::unspendable_account::UnspendableAccount::from_secret(secret).account_id;
392 |
393 |     let unspendable_account_bytes_digest = digest_felts_to_bytes(unspendable_account);
394 |     let unspendable_account_bytes: [u8; 32] = unspendable_account_bytes_digest
395 |         .as_ref()
396 |         .try_into()
397 |         .expect("BytesDigest is always 32 bytes");
398 |     let unspendable_account_id = SubxtAccountId(unspendable_account_bytes);
399 |
400 |     Ok((secret, unspendable_account_id, unspendable_account))
401 | }
```

Another repeats the same conversion inline:

wormhole/example/src/main.rs:606-615

```
606 |     let unspendable_account =
607 |
wormhole_circuit::unspendable_account::UnspendableAccount::from_secret(secret).account_id;
608 |
609 |     // Convert Digest (field elements) to BytesDigest (bytes)
610 |     let unspendable_account_bytes_digest = digest_felts_to_bytes(unspendable_account);
611 |     let unspendable_account_bytes: [u8; 32] = unspendable_account_bytes_digest
612 |         .as_ref()
613 |         .try_into()
614 |         .expect("BytesDigest is always 32 bytes");
615 |     let unspendable_account_id = SubxtAccountId(unspendable_account_bytes);
```

A third uses `UnspendableAccount::from_secret(...).account_id` directly without converting to `SubxtAccountId` at all:

wormhole/example/src/main.rs:168-170

```
168 |         let unspendable_account =
169 |             wormhole_circuit::unspendable_account::UnspendableAccount::from_secret(secret)
170 |                 .account_id;
```

The `get_unspendable_account` helper is only called once and does nothing beyond what `UnspendableAccount::from_secret` already provides, plus the `Digest` → `SubxtAccountId` boilerplate. The inline copy at line 606 duplicates the same boilerplate without using the helper.

Recommendations

- Remove unnecessary utils and take advantage of the existing boilerplate consistently.

EQ-QNT-WORMHOLE-R-09: Wormhole pallet weights are stale

The weight file for `pallet_wormhole` has not been updated to reflect the current extrinsic interface. It contains a benchmarked weight for a single-proof verification path that no longer exists, and a hand-written placeholder for the aggregated proof path that replaced it.

The file header claims it was auto-generated on 2025-11-27, but the actual changes were made manually in commit `fc80f03` on 2026-02-15 as part of the plonky2 upgrade. The single-proof weight was adjusted by hand (from 17.1 ms to 14.3 ms) and the aggregated weight was added as a placeholder in the same commit. The auto-generated header is misleading.

Single-proof weight is dead code

`verify_wormhole_proof` no longer exists in the pallet's call interface:

pallets/wormhole/src/weights.rs:72-79

```
fn verify_wormhole_proof() -> Weight {
  // Proof Size summary in bytes:
  //  Measured:  `256`
  //  Estimated: `3593`
  // Minimum execution time: 14_318_000_000 picoseconds.
  Weight::from_parts(14_468_000_000, 3593)
    .saturating_add(T::DbWeight::get().reads(4_u64))
    .saturating_add(T::DbWeight::get().writes(4_u64))
```

The weight function, trait definition, and benchmark setup for it remain in the code but are unused.

Aggregated-proof weight was never benchmarked

The weight for `verify_aggregated_proof` is a manual estimate – 8× the old single-proof weight:

pallets/wormhole/src/weights.rs:82-92

```
/// Weight for verifying an aggregated proof.
/// This is a placeholder - proper benchmarking should be done.
/// Aggregated proofs verify multiple transfers at once, so the weight
/// should be higher than single proof verification.
fn verify_aggregated_proof() -> Weight {
  // Placeholder: use 8x the single proof weight as a conservative estimate
  // for the default configuration (8 leaf proofs)
  Weight::from_parts(115_744_000_000, 28744)
    .saturating_add(T::DbWeight::get().reads(32_u64))
    .saturating_add(T::DbWeight::get().writes(32_u64))
}
```

The 8× multiplier matches the number of leaf proofs in the default aggregation configuration, so the intent could have been to account for verifying 8 proofs worth of work. It may also have been chosen as an arbitrary conservative safety margin. Either way, plonky2 verification does not scale this way – the verifier checks one fixed-structure proof regardless of how many leaves were aggregated. The cost difference between single and aggregated verification comes from additional public input processing and storage I/O, not from re-doing per-leaf work. The placeholder is likely an overestimate, though the degree depends on the target hardware.

The benchmark infrastructure in `benchmarking.rs` is wired up correctly for the aggregated path, but depends on a proof fixture (`aggregated_proof.hex`) that was a single-byte placeholder (`00`) at the time of review. Running the benchmark requires first generating a valid fixture via `quantus developer build-circuits` and the aggregation toolchain.

The storage I/O components (32 reads, 32 writes) are also hardcoded rather than derived from the actual number of exit accounts and nullifiers in the batch.

Impact

The block compute budget is 6 seconds (75% available for normal dispatches). An overestimated weight means the chain reserves more capacity per wormhole extrinsic than it needs, reducing the number of aggregated proofs verifiable per block. Since this is the only path for wormhole transactions, the overestimate directly limits throughput. This is not a security issue – overestimating is the conservative direction and cannot cause block time overruns.

Recommendations

- Generate a valid aggregated proof fixture and run `frame-benchmarking` to produce measured weights for `verify_aggregated_proof`.
- Remove the `verify_wormhole_proof` weight function and trait method.
- Consider making the storage I/O weight components scale with the actual number of exit accounts and nullifiers.

EQ-QNT-WORMHOLE-O-01: Wormhole Test Coverage

The pallet and the aggregator circuit both have gaps in test coverage. This finding catalogs what is tested, what is not, and what is broken.

Pallet: untested `verify_aggregated_proof`

All four pallet tests exercise `record_transfer`, the helper that records proofs and emits events. The pallet's only extrinsic, `verify_aggregated_proof`, has no test coverage. Proof deserialization, nullifier checks, block hash validation, volume fee math, and minimum amount enforcement are all untested. Once a happy-path test exists, negative cases should follow: duplicate nullifiers, invalid proofs, below-minimum amounts, missing block hashes, and missing digests, each asserting the correct error is returned.

Pallet: fee split untested

The 50% burn / 50% miner split for volume fees is never exercised. This is the code path where EQ-QNT-WORMHOLE-F-03 (incorrect preimage handling) silently burns miner rewards, which went undetected because nothing tests it.

No end-to-end prove → aggregate → verify coverage in CI

The full chain from proof generation through aggregation to on-chain verification is tested in two places, both ignored:

- `verify_aggregated_proof_with_prebuilt_verifier` in `aggregator_tests.rs` is marked `#[ignore = "requires regenerating circuit binaries after 2-output layout change"]`.
- The integration tests in `quantus-cli/tests/wormhole_integration.rs` call the pallet's `verify_aggregated_proof` extrinsic but require a running local node and are also `#[ignore]`.

The non-ignored aggregator tests call `circuit_data.verify()` on the in-memory circuit built during the test. This checks that the proof is valid for that circuit, but does not verify binary compatibility between the aggregation circuit and the prebuilt verifier binaries embedded in the pallet. A change that causes the two to diverge would not be caught.

Aggregator: missing circuit-level test cases

The aggregator circuit tests (`tree.rs`, `layer1.rs`) cover the happy path and two negative cases (different blocks, mismatched `asset_id`). Several cases are missing:

- **All-dummy batch.** An aggregation where every proof is a dummy can still be built. No test verifies that the pallet rejects or correctly no-ops such a proof.
- **Dummy nullifier replacement.** The existing mixed-dummy test checks that aggregation succeeds but does not inspect whether dummy nullifiers were actually replaced with random values.
- **Mismatched `volume_fee_bps`.** The circuit enforces matching `volume_fee_bps` across proofs, but unlike `asset_id` there is no negative test for a mismatch.
- **Exit-sum overflow.** Exit amounts are range-checked to 32 bits. No test exercises boundary values or checks that an overflow is rejected.

Duplicated `account_id` helper

The mock defines its own `account_id` helper:

pallets/wormhole/src/mock.rs:29-34

```
29 | /// Helper function to convert a u64 to an AccountId32
30 | pub fn account_id(id: u64) -> AccountId {
31 |  let mut bytes = [0u8; 32];
32 |  bytes[0..8].copy_from_slice(&id.to_le_bytes());
33 |  AccountId::new(bytes)
34 | }
```

The `mining-rewards` mock has the same thing (see EQ-QNT-MINING-R-01). Once miner reward distribution is tested here, both mocks will need compatible account/preimage helpers and should share them.

Unexplained `MintingAccount` constant

pallets/wormhole/src/mock.rs:118-121

```
118 | pub const MintingAccount: AccountId = AccountId::new([
119 |   231, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
120 |   0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
121 | ]);
```

The byte array `[231, 3, 0, ...]` (999 in little-endian u16) has no comment explaining the choice.

Recommendation

- Add happy-path and error-branch tests for `verify_aggregated_proof`.
- Add the missing aggregator negative/edge-case tests listed above.
- Add a CI step that generates circuit binaries from scratch (twice, to simulate independent circuit-builder runs), then runs the full prove → aggregate → verify chain against them. This catches binary-compatibility regressions without depending on checked-in binaries or a running node.
- Extract shared test utilities (e.g. `account_id`) into a common module.
- Document or simplify the `MintingAccount` constant.

EQ-QNT-WORMHOLE-O-02: MAX_PROOF_LEN Storage Proof Constraint Undocumented

The wormhole circuit constrains Merkle-Patricia trie storage proof depth to a maximum of 20 nodes:

wormhole/circuit/src/storage_proof/mod.rs:24-24

```
24 | pub const MAX_PROOF_LEN: usize = 20;
```

Context

The circuit verifies that a `TransferProof` entry exists in the blockchain's state trie by checking a Merkle-Patricia proof path from the block's `state_root` to the storage entry. The path length (number of nodes traversed) depends on how deep the storage key sits in the trie.

The constraint exists because the circuit pre-allocates fixed-size arrays for proof verification:

wormhole/circuit/src/storage_proof/mod.rs:45-51

```
        // Setup targets. Each 8-bytes are represented as their equivalent field element. We also
        // need to track total proof length to allow for variable length.
        let proof_data: Vec<_> = (0..MAX_PROOF_LEN)
            .map(|_| builder.add_virtual_targets(PROOF_NODE_MAX_SIZE_F))
            .collect();

        let indices: Vec<_> = (0..MAX_PROOF_LEN)
```

If a proof exceeds this length, it is rejected during proof generation (witness filling):

wormhole/circuit/src/storage_proof/mod.rs:262-269

```
        // bail if proof is too long
        if self.proof.len() > MAX_PROOF_LEN {
            bail!(
                "proof length exceeds maximum allowed length: {} > {}",
                self.proof.len(),
                MAX_PROOF_LEN
            );
        }
```

However, this check is a prover-side guard in `fill_targets`, not a constraint enforced by the ZK circuit itself. The circuit iterates over exactly `MAX_PROOF_LEN` slots and uses the witness-provided `proof_len` value to determine which nodes to validate and where the leaf is:

wormhole/circuit/src/storage_proof/mod.rs:182-194

```
        for i in 0..MAX_PROOF_LEN {
            let node = &proof_data[i];

            // Check if this is a valid proof node or a dummy one.
            let is_proof_node = is_const_less_than(builder, i, proof_len, n_log);

            // Check if this is a leaf node.
            let i_t = builder.constant(F::from_canonical_usize(i));
            let is_leaf_node = builder.is_equal(i_t, proof_len);

            // Compute the hash of this node and compare it against the previous hash.
            // Only enforce validation for non-dummy proofs (output_amount > 0).
            let should_validate_node = builder.mul(is_proof_node.target, is_not_dummy.target);
```

If `proof_len` were set to a value $\geq$ `MAX_PROOF_LEN` (e.g. 20), then `is_leaf_node` (which checks `i == proof_len`) would never be true for any loop index $i \in \{0, ..., 19\}$. The leaf validation that ties the storage entry to the claimed transfer data would be skipped entirely. Any storage proof with a valid node hash chain from the root would then be accepted regardless of its leaf content.

Practical Impact

In reality this is not exploitable: the `proof_len` overflow would still require all `MAX_PROOF_LEN` nodes to form a valid hash chain from the state root. Security comes from the fact that such path would not occur in practice.

Storage proof depth grows logarithmically with the number of entries in the state trie. Since storage keys are hashes, an attacker cannot force deeper paths by choosing keys. But because keys are uniformly random, reaching depth 20 is a birthday-problem event: two keys must share the same first 20 nibbles. Applying the birthday bound at 50% probability gives approximately $1.2 \cdot \sqrt{16^{20}} = 1.2 \cdot 16^{10} \approx 1.32 \times 10^{12}$ entries before this becomes likely. Real-world Substrate chains typically have proof depths of 5–15 nodes. Test data in the repository uses proof lengths of 7–8 nodes.

Missing Documentation

QIP-5 does not document this circuit-level limitation or the security assumptions that make the `proof_len` overflow benign.

Recommendations

- Add a circuit constraint `proof_len < MAX_PROOF_LEN`, eliminating reliance on the prover-side guard in `fill_targets` and making the bound explicit in the verified circuit.
- Document `MAX_PROOF_LEN = 20` in QIP-5's technical constraints section.
- Document the security assumptions: that proof depth is bounded by the hash-based key distribution, and that the hash chain from the state root is what prevents forgery.

EQ-QNT-WORMHOLE-O-03: Dummy Proof Nullifiers Written to Permanent Storage

Aggregated batches are padded with dummy proofs when fewer than N real proofs are available. The pallet filters dummy `account_data` entries but writes all N nullifiers, including dummy ones, to permanent storage.

Dummy proof design

The leaf circuit skips validation when `block_hash == [0; 32]` and both output amounts are zero:

wormhole/circuit/src/circuit.rs:142-166

```
        // Dummy proof detection: requires BOTH block_hash == 0 AND output_amounts == 0.
        // This prevents an attacker from slipping funds through with a zero block hash
        // but positive output amounts.
        let zero = builder.zero();
        let one = builder.one();
```

```
147 |
148 |         // Check if all four limbs of block_hash are zero
149 |         let bh = &targets.block_header.block_hash.elements;
150 |         let bh0_is_zero = builder.is_equal(bh[0], zero);
151 |         let bh1_is_zero = builder.is_equal(bh[1], zero);
152 |         let bh2_is_zero = builder.is_equal(bh[2], zero);
153 |         let bh3_is_zero = builder.is_equal(bh[3], zero);
154 |
155 |         let bh01_zero = builder.and(bh0_is_zero, bh1_is_zero);
156 |         let bh23_zero = builder.and(bh2_is_zero, bh3_is_zero);
157 |         let block_hash_is_zero = builder.and(bh01_zero, bh23_zero);
158 |
159 |         // Check if both output amounts are individually zero
160 |         let leaf = &targets.storage_proof.leaf_inputs;
161 |         let output_1_is_zero = builder.is_equal(leaf.output_amount_1, zero);
162 |         let output_2_is_zero = builder.is_equal(leaf.output_amount_2, zero);
163 |         let both_outputs_zero = builder.and(output_1_is_zero, output_2_is_zero);
164 |
165 |         // is_dummy = block_hash_is_zero AND both_outputs_zero
166 |         let is_dummy = builder.and(block_hash_is_zero, both_outputs_zero);
```

Dummies use `exit_account = [0; 32]`, `output_amount = 0`, and prover-chosen nullifiers. The aggregator picks random nullifiers to avoid intra-batch collisions:

wormhole/aggregator/src/dummy_proof.rs:40-68

```
 40 |
 41 | /// Sentinel block hash for dummy proofs (all zeros).
 42 | /// The leaf circuit skips all validation when block_hash == 0.
 43 | pub const DUMMY_BLOCK_HASH: [u8; 32] = [0u8; 32];
 44 |
 45 | /// Exit account used by dummy proofs (all zeros).
 46 | /// Dummies form their own exit account group but contribute 0 to the sum.
 47 | pub const DUMMY_EXIT_ACCOUNT: [u8; 32] = [0u8; 32];
 48 |
 49 | // ============================================================================
 50 | // Internal constants for dummy proof generation
 51 | // ============================================================================
 52 |
 53 | const DEFAULT_SECRET: &str =
"4c8587bd422e01d961acdc75e7d66f6761b7af7c9b1864a492f369c9d6724f05";
 54 | const DEFAULT_TRANSFER_COUNT: u64 = 4;
 55 | const DEFAULT_FUNDING_ACCOUNT: [u8; 32] = [
 56 |     226, 124, 203, 9, 80, 60, 124, 205, 165, 5, 178, 216, 195, 15, 149, 38, 116, 1, 238, 133,
181,
 57 |     154, 106, 17, 41, 228, 118, 179, 82, 141, 225, 76,
 58 | ];
 59 | const DEFAULT_INPUT_AMOUNT: u32 = 100;
 60 | const DEFAULT_OUTPUT_AMOUNT: u32 = 0;
 61 | const DEFAULT_VOLUME_FEE_BPS: u32 = 10;
 62 |
 63 | // Block data - all zeros for dummy sentinel
 64 | const DEFAULT_PARENT_HASH: [u8; 32] = [0u8; 32];
 65 | const DEFAULT_BLOCK_NUMBER: u32 = 0;
 66 |
```

```
// Nullifier is all zeroes as well. During aggregation, we replace the dummy nullifier targets with random nullifiers to prevent deduplication of dummy proofs.
const DEFAULT_NULLIFIER: [u8; 32] = [0u8; 32];
```

The leaf circuit's nullifier hash check is conditionally skipped for dummies, so the nullifier value is unconstrained. The aggregation circuit injects it as a private witness.

On-chain processing

The pallet skips dummy `account_data` entries:

pallets/wormhole/src/lib.rs:316-328

```
    for (idx, account_data) in aggregated_inputs.account_data.iter().enumerate() {
      // Skip dummy account slots (exit_account == 0 with zero amount)
      // Dummy proofs from aggregation padding have all-zero exit accounts
      // Also skip deduplicated slots (the circuit zeros out duplicate exit accounts)
      let exit_account_bytes: [u8; 32] =
        (*account_data.exit_account).as_ref().try_into().map_err(|e| {
          log::error!("Failed to convert exit_account at idx {}: {:?}", idx, e);
          Error::<T>::InvalidAggregatedPublicInputs
        })?;

      if exit_account_bytes == [0u8; 32] || account_data.summed_output_amount == 0 {
        continue;
      }
```

The nullifier loop, however, writes all N unconditionally:

pallets/wormhole/src/lib.rs:266-276

```
    for nullifier in &aggregated_inputs.nullifiers {
      let nullifier_bytes: [u8; 32] = (*nullifier)
        .as_ref()
        .try_into()
        .map_err(|_| Error::<T>::InvalidAggregatedPublicInputs)?;
      ensure!(
        !UsedNullifiers::<T>::contains_key(nullifier_bytes),
        Error::<T>::NullifierAlreadyUsed
      );
      UsedNullifiers::<T>::insert(nullifier_bytes, true);
    }
```

This could be a defensive measure: filtering dummy nullifiers would let observers count `UsedNullifiers` insertions per batch and derive the exact number of real proofs. The calldata already leaks an approximation though: the number of non-zero `account_data` slots equals the number of unique exit addresses in the batch, a lower bound on real proof count. Writing all N nullifiers only adds privacy beyond that if users deliberately route multiple proofs to the same

exit address, pushing the visible unique-exit count below the actual real-proof count. That is a conceivable design intent, though a weak and indirect one. If it is the intent, it is not documented anywhere.

If it is not intentional, the writes are unnecessary. With N=8 and K real proofs per batch, each batch writes 8-K dummy entries that serve no functional purpose and accumulate indefinitely. At full block saturation all nullifiers — real and dummy — consume up to 109 GiB per year (see Section 12.).

Nullifier Bloat Attack cost

Each aggregated proof requires at least one real proof, and the pallet enforces a minimum total exit amount of 10 tokens (`WormholeMinimumTransferAmount`). An attacker maximising dummy-nullifier writes would submit K=1 real proof per batch, recycling the same 10 tokens: deposit, prove, exit (recovering 9.99 tokens after the 10 bps volume fee), then deposit again. The ongoing cost per batch is therefore 0.01 tokens in fees (burned or distributed), not the full transfer amount. At the maximum rate of 37 batches per block, sustaining full-block saturation for a year would consume roughly 970k tokens in volume fees — approximately 4.6% of the 21 million token maximum supply.

On an Apple M2, generating one leaf proof takes approximately 1.5 seconds and aggregating the batch of 8 takes a further 10.5 seconds — a total of 12 seconds per batch, which matches the block time. A single machine therefore sustains approximately one batch per block. Saturating all 37 available slots per block requires 37 concurrent workers.

Attacker-chosen nullifiers

Since dummy nullifiers are unconstrained by the circuit, a malicious aggregator has full control over them. However, there is no meaningful way to exploit this. Pre-registering another user's nullifier to block their withdrawal would require knowing the user's wormhole secret (from which the nullifier is derived), at which point the attacker could steal the funds directly.

Recommendations

- Document whether writing dummy nullifiers is a deliberate privacy measure or an oversight, and act accordingly: either add a code comment explaining the rationale, or filter dummy nullifiers before the storage write loop.

EQ-QNT-WORMHOLE-O-04: QIP-5 describes parent_hash-based cross-block aggregation that was never implemented

QIP-5 says proof aggregation works across contiguous blocks, with `parent_hash` chaining them together. The implementation does something different: all leaf proofs must use the same *proof-generation block*, but since `TransferProof` entries persist in the state trie indefinitely, transfers from earlier blocks can still be batched by proving them against a later block's `state_root`. The spec was never updated.

QIP-5 claims

The `parent_hash` public input description says it exists for cross-block aggregation:

qip-0005.md:37-38

> - `parent_hash`: The parent hash of the transaction's block. We expose this publicly so proof aggregators can efficiently verify the connection between storage proofs generated relative to adjacent blocks.
> - `block_number`: The block height corresponding to `block_hash`. This is public so the verifier/aggregator can check that it matches the associated header and to know which blocks to connect when aggregating proofs.

The spec also says aggregation is restricted to contiguous blocks because the circuit verifies connectivity via `parent_hash`:

qip-0005.md:88-88

> It's important to note that we only support aggregation of transfer proofs across contiguous blocks. The reason being is that the circuit verifies connectivity of deduplicated blocks via parent-hash and doesn't account for gaps for the purposes of keeping the circuit constraints low. New sets of aggregated proofs must be created for transfer proofs from non-contiguous blocks.

What actually happens

The aggregation circuit requires all real proofs to share the same `block_hash`. No parent_hash chaining exists. The circuit comments acknowledge this directly:

wormhole/aggregator/src/circuits/tree.rs:233-241

```
    // BLOCK VALIDATION (Fixed Structure)
    // ==================================================================
    // All real proofs (block_hash != 0) must reference the same block for their storage proofs.
    // This means all proofs must be generated against the same chain state snapshot.
    // Note: The underlying transfers can occur in different blocks; this constraint only
    // applies to the block used when generating the storage proof (when the proof is created).
    //
    // We use the first real proof's block as the reference.
    // Dummies (block_hash == 0) are skipped via conditional constraints.
```

wormhole/aggregator/src/circuits/tree.rs:255-272

```
    // For each proof, check: if it's not a dummy, it must match block_ref
    // Constraint: is_dummy OR (block_hash == block_ref)
    // Equivalently: NOT(is_real AND block_hash != block_ref)
    for i in 0..n_leaf {
        let block_i = limbs4_at_offset::<LEAF_PI_LEN, BLOCK_HASH_START>(child_pi_targets, i);

        // is_dummy_i = (block_i == [0,0,0,0])
        let is_dummy_i = bytes_digest_eq(&mut builder, block_i, dummy_sentinel);
        is_dummy_flags.push(is_dummy_i);

        // matches_ref = (block_i == block_ref)
        let matches_ref = bytes_digest_eq(&mut builder, block_i, block_ref);

        // Constraint: is_dummy OR matches_ref must be true
        // i.e., is_dummy + matches_ref - is_dummy*matches_ref >= 1
        // Since both are bool, OR = is_dummy + matches_ref - is_dummy*matches_ref
        let or_result = builder.or(is_dummy_i, matches_ref);
        builder.connect(or_result.target, one);
```

`PARENT_HASH_START`, which previously existed in the aggregation circuit, has been removed. `parent_hash` is still a private input in each leaf proof (`inputs.rs:45-46`) – the leaf circuit needs it to compute `block_hash` from the header preimage – but the aggregation circuit never reads, constrains, or forwards it.

Cross-block batching works anyway

The same-block constraint is less restrictive than it sounds. `TransferProof` is a persistent `StorageMap` with no expiration. A transfer recorded in block X remains in the state trie at block Y (for any Y > X). A prover can:

0. Pick any block Y newer than all desired transfers
1. Generate storage proofs for transfers from blocks $X_1, X_2, \ldots, X_n$ against Y's `state_root`
2. Aggregate all leaf proofs with `block_hash = hash(Y)`

No contiguity requirement, no parent_hash chaining – any set of historical transfers can be batched as long as one block exists after all of them.

Where `parent_hash` actually matters

Only inside a single leaf proof. The circuit hashes `(parent_hash, block_number, state_root, extrinsics_root, digest)` and checks the result equals `block_hash`. This binds header fields to the public `block_hash`, but the pallet never reads `parent_hash` itself.

Impact

QIP-5 describes a mechanism (parent_hash chaining) that does not exist, and does not describe the mechanism that does (state persistence). Anyone reading the spec would think cross-block batching requires contiguous blocks and parent_hash verification. In practice, the only requirement is a shared proof-generation block.

Recommendations

- Update QIP-5: remove the parent_hash chaining claims and document the actual design. All leaf proofs use the same proof-generation block; cross-block batching is possible because `TransferProof` entries persist.

EQ-QNT-CHAIN-R-01: Outdated trie-db Dependency Triggers Rust 2024 Edition Warning

The chain workspace uses `trie-db = "0.30.0"`, which triggers never type fallback warnings that will become hard errors in Rust 2024 edition.

Issue

The workspace Cargo.toml specifies an outdated version:

Cargo.toml:122-122

```
122 | trie-db = { version = "0.30.0", default-features = false }
```

This version relies on never type (`!`) fallback behaviour that changed between Rust editions. The Rust compiler issues warnings about code that will break in the 2024 edition:

```
warning: this function depends on never type fallback being `()`
  --> trie-db related code
   |
```

```
  = warning: this was previously accepted by the compiler but is being phased out
  = note: for more information, see rust-lang/rust#123748
  = help: specify the type explicitly
```

Impact

- Current code compiles with warnings
- Migration to Rust 2024 edition will be blocked until resolved

Solution

Upgrade to `trie-db = "0.31.0"` which addresses the never type fallback issues:

```
[workspace.dependencies]
trie-db = { version = "0.31.0", default-features = false }
```

Recommendation

1. Update workspace dependency to `trie-db = "0.31.0"`
4. Consider updating related crates (`trie-root`, `trie-bench`) simultaneously

EQ-QNT-WORMHOLE-I-02: Fee Formula Produces Zero Fee at `volume_fee_bps = 10000`

The on-chain fee formula uses `saturating_sub` and `checked_div` to compute the fee from the exit amount:

pallets/wormhole/src/lib.rs:364-367

```
364 |      let total_fee_u128 = total_exit_u128
365 |        .saturating_mul(fee_bps)
366 |        .checked_div(10000u128.saturating_sub(fee_bps))
367 |        .unwrap_or(0);
```

When `volume_fee_bps = 10000`, `10000u128.saturating_sub(10000)` evaluates to zero. `checked_div(0)` returns `None`, and `unwrap_or(0)` silently yields a fee of zero. Taken in isolation this is a logic error: a 100% fee rate should not produce zero fee.

Circuit constraint makes this code path unreachable

The leaf circuit range-checks `fee_complement = 10000 - volume_fee_bps` to 14 bits:

wormhole/circuit/src/storage_proof/mod.rs:156-159

```
156 |         let fee_complement = builder.sub(ten_thousand, leaf_inputs.volume_fee_bps); // 10000 -
volume_fee_bps
157 |
158 |         // Constrain fee_complement to 14 bits, implying volume_fee_bps <= 10000 and
fee_complement < 2^14.
159 |         builder.range_check(fee_complement, 14);
```

This enforces `volume_fee_bps ≤ 10000` in every proof. At exactly 10000, `fee_complement = 0`, so the fee constraint reduces to:

$$(\text{output}_1 + \text{output}_2) \times 10000 \leq \text{input} \times 0$$

which forces both output amounts to zero. Any proof with `volume_fee_bps = 10000` is only circuit-valid when the total exit amount is zero.

Minimum-amount gate

Zero output amounts mean `total_exit_amount = 0`, which fails the minimum transfer check before the fee formula is reached:

pallets/wormhole/src/lib.rs:348-351

```
348 |      ensure!(
349 |        total_exit_amount >= T::MinimumTransferAmount::get(),
350 |        Error::<T>::TransferAmountBelowMinimum
351 |      );
```

The runtime rejects the transaction with `TransferAmountBelowMinimum`. The zero-fee code path is therefore unreachable under any valid proof.

Practical significance

Setting `VolumeFeeRateBps = 10000` via governance would silently brick the wormhole: all proofs would be rejected, but with a misleading error rather than an invalid-fee-rate error. In practice this configuration has no rational use case — a 100% protocol fee leaves nothing for the user — and is not a credible governance outcome.

5. OTHER OBSERVATIONS

EQ-QNT-MISC-R-01: Unnecessary Use of Clone

Several places in the codebase clone values where a borrow would suffice. This finding collects instances identified during the audit.

- `mining-rewards` pallet: `mint_reward` accepts `Option<T::AccountId>` by value, forcing the caller to `clone` the miner account:

pallets/mining-rewards/src/lib.rs:175-179

```
175 |     // Send fees to miner if any
176 |     Self::mint_reward(miner.clone(), tx_fees);
177 |
178 |     // Send block rewards to miner
179 |     Self::mint_reward(miner, miner_reward);
```

The function takes ownership but never needs it:

pallets/mining-rewards/src/lib.rs:229-229

```
229 |   fn mint_reward(maybe_miner: Option<T::AccountId>, reward: BalanceOf<T>) {
```

Changing the parameter to `Option<&T::AccountId>` and calling with `miner.as_ref()` avoids the clone.

- `mining-rewards` pallet: in `mint_reward`, `miner` is cloned twice — once when passed to `record_transfer_proof` and again, unnecessarily, in the `MinerRewarded` event

pallets/mining-rewards/src/lib.rs:243-243

```
243 |         miner.clone(),
```

pallets/mining-rewards/src/lib.rs:247-247

```
247 |       Self::deposit_event(Event::MinerRewarded { miner: miner.clone(), reward });
```

- `wormhole` pallet: in `record_transfer`, clones can be replaced with borrows since `StorageMap::insert` accepts references:

pallets/wormhole/src/lib.rs:497-500

```
    TransferProof::<T>::insert(
      (asset_id.clone(), current_count, from.clone(), to.clone(), amount),
      (),
    );
```

Recommendation

Prefer borrowing over cloning when the callee does not need ownership.

EQ-QNT-MISC-R-03: Overly Broad `pub` Visibility across ZK Circuit Crates

Many items across the `qp-zk-circuits` workspace are marked `pub` despite being internal implementation details. This inflates the public API surface, making it harder to reason about crate boundaries and to refactor safely. Each unnecessary `pub` adds to the API surface a consumer must understand and the maintainer must preserve across versions. It also causes rustdoc to generate documentation for items that are not intended for external use.

A representative example is `BinaryHashes::hash_file`, which is only called by `BinaryHashes::from_directory` within the same module:

wormhole/aggregator/src/config.rs:33-40

```
    /// Compute SHA256 hash of a file and return as hex string
    pub fn hash_file<P: AsRef<Path>>(path: P) -> Result<String> {
        use sha2::{Digest, Sha256};
        let bytes = std::fs::read(path.as_ref())
            .map_err(|e| anyhow!("Failed to read {}: {}", path.as_ref().display(), e))?;
        let hash = Sha256::digest(&bytes);
        Ok(hex::encode(hash))
    }
```

Other instances identified during the audit:

- `load_dummy_proof` — called only within `aggregator.rs`, not re-exported from the crate root, yet marked `pub`:

wormhole/aggregator/src/dummy_proof.rs:101-107

```
/// Deserialize a dummy proof from raw bytes.
pub fn load_dummy_proof(
    bytes: Vec<u8>,
    common_data: &CommonCircuitData<F, D>,
) -> anyhow::Result<ProofWithPublicInputs<F, C, D>> {
    ProofWithPublicInputs::<F, C, D>::from_bytes(bytes, common_data)
}
```

- `generate_random_nullifier` — used only in `circuits/tree.rs`, placed under an "Internal implementation" comment, yet marked `pub`:

wormhole/aggregator/src/dummy_proof.rs:183-190

```
183 | /// Generate a random 32-byte nullifier for dummy proofs.
184 | pub fn generate_random_nullifier() -> BytesDigest {
185 |     let mut rng = rand::thread_rng();
186 |     let mut nullifier = [0u8; 32];
187 |     rng.fill(&mut nullifier);
188 |     // If the bytes digest conversion fails (extremely unlikely), just regenerate until we get
a valid one.
189 |     BytesDigest::try_from(nullifier).unwrap_or_else(|_| generate_random_nullifier())
190 | }
```

- `BinaryHashes` and all its fields are `pub`, but the struct is only used internally by `CircuitBinsConfig`. No external consumer needs direct access to individual hash fields.

- `WormholeProofAggregator` exposes `leaf_circuit_data`, `config`, and `proofs_buffer` as `pub` fields while `dummy_proof_template` is private. The public fields are only accessed in tests; they are not part of a meaningful API contract:

wormhole/aggregator/src/aggregator.rs:29-37

```
29 | pub struct WormholeProofAggregator {
30 |     pub leaf_circuit_data: VerifierCircuitData<F, C, D>,
31 |     pub config: AggregationConfig,
32 |     pub proofs_buffer: Option<Vec<ProofWithPublicInputs<F, C, D>>>,
33 |     /// A single dummy proof template compatible with this aggregator's circuit.
34 |     /// Cloned as many times as needed for padding during `aggregate()`.
35 |     /// Each clone gets a unique nullifier to avoid duplicate nullifier errors on-chain.
36 |     dummy_proof_template: ProofWithPublicInputs<F, C, D>,
37 | }
```

- `aggregate_proofs` and `Layer1Wrapper` in `circuits/` are internal aggregation machinery, called only by `WormholeProofAggregator::aggregate`.

- In `common/src/gadgets.rs`, `range32` and `count_unique_4x32_keys` are `pub` but unused anywhere in the codebase. `xor` is `pub` but called only within the same file.

Recommendation

Apply the principle of least visibility: default to private and widen to `pub(crate)` or `pub` only where cross-crate access is required. Remove dead public functions (`range32`, `count_unique_4x32_keys`). For struct fields that tests need, consider `#[cfg(test)] pub` or test-specific accessor methods.

EQ-QNT-MISC-O-01: Mining Rewards and Wormhole minting accounts are derived differently

The mining-rewards and wormhole pallets each define a separate `MintingAccount` used as the "from" field in transfer proof records. They are derived differently:

runtime/src/configs/mod.rs:169-169

```
169 |   pub const MintingAccount: AccountId = AccountId::new([1u8; 32]);
```

runtime/src/configs/mod.rs:687-687

```
687 |  pub WormholeMintingAccount: AccountId = PalletId(*b"wormhole").into_account_truncating();
```

The first is a fixed byte pattern, the second is derived from a `PalletId`. The difference is not commented on in the codebase.

EQ-QNT-MISC-R-02: Unused Runtime Constants

The runtime configuration defines several `parameter_types!` constants that are not referenced anywhere in the codebase.

Dead code

The following constants are defined but never used:

- `MaxTokenAmount` (line 125)
- `DefaultMintAmount` (line 126)
- `MinimumDeposit` (line 211)
- `NoPreimagePostponement` (line 397)

Technical committee referenda constants shadowed by community ones

Six constants were defined for the technical committee referenda instance:

runtime/src/configs/mod.rs:331-343

```
331 | parameter_types! {
332 |  // Default voting period (28 days)
333 |  pub const TechReferendumDefaultVotingPeriod: BlockNumber = 28 * DAYS;
334 |  // Minimum time before a successful referendum can be enacted (4 days)
335 |  pub const TechReferendumMinEnactmentPeriod: BlockNumber = 4 * DAYS;
336 |  // Maximum number of active referenda
337 |  pub const TechReferendumMaxProposals: u32 = 100;
338 |  // Submission deposit for referenda
```

```
pub const TechReferendumSubmissionDeposit: Balance = 100 * UNIT;
// Undeciding timeout (90 days)
pub const TechUndecidingTimeout: BlockNumber = 45 * DAYS;
pub const TechAlarmInterval: BlockNumber = 1;
}
```

None of them are referenced. The `Config<TechReferendaInstance>` implementation uses the community referenda constants instead (`ReferendumSubmissionDeposit`, `ReferendumMaxProposals`, `UndecidingTimeout`, `AlarmInterval`):

runtime/src/configs/mod.rs:376-382

```
type SubmissionDeposit = ReferendumSubmissionDeposit;
/// Maximum number of referenda that can be in the deciding phase simultaneously.
type MaxQueued = ReferendumMaxProposals;
/// Time period after which an undecided referendum will be automatically rejected.
type UndecidingTimeout = UndecidingTimeout;
/// The frequency at which the pallet checks for expired or ready-to-timeout referenda.
type AlarmInterval = AlarmInterval;
```

As a result, both referenda instances share identical governance parameters. Whether this is intentional is not documented.

EQ-QNT-ZK-I-01: NEON SIMD Poseidon Code Not Validated in This Audit

The Poseidon hash implementation for the Goldilocks field includes hand-written NEON SIMD intrinsics (AArch64) that accelerate the `sbox_layer` and `mds_layer` operations. These are the two most performance-sensitive functions in the Poseidon permutation. On AArch64 targets with NEON support, they replace the generic Rust implementations via `#[cfg]` gates and execute inside `unsafe` blocks.

This code is carried over from upstream plonky2 and was not modified by Quantus. A separate AVX2 (x86_64) variant exists in the source tree but is entirely commented out and not compiled.

The NEON code paths were not validated for correctness against the generic fallback during this audit. Any discrepancy between the SIMD and scalar implementations would produce different Poseidon digests on different architectures, which could lead to consensus failures if nodes running on AArch64 and x86_64 disagree on hashes.

Recommendation

No action required. This is recorded for completeness. If Quantus nodes will run on AArch64 hardware in production, consider adding cross-architecture consistency tests that verify the

NEON-accelerated Poseidon output matches the scalar implementation for a set of known test vectors.

EQ-QNT-ZK-O-02: Dual Hash Strategy: Poseidon for FRI, Poseidon2 for Application Circuits

The plonky2 fork uses two independent Poseidon-family hash functions. The original Poseidon, via `PoseidonGoldilocksConfig`, serves as the proof-system hash for FRI polynomial commitments and Merkle trees. Poseidon2 was added by Quantus for application-level hashing and is called explicitly in circuit code via `Poseidon2Hash::hash_no_pad` and `CircuitBuilder::hash_n_to_hash_no_pad_p2`.

Poseidon2 is used for all application-level hashing across both major circuit families:

- **Voting**: public key commitments, vote commitments, and voter eligibility Merkle trees.
- **Wormhole**: nullifier derivation (double-hash), unspendable account derivation (double-hash), block header hashing, and storage trie proof verification.
- **Common**: substrate storage trie node hashing via `qp_poseidon_core::hash_padded_bytes`.

Both hashes operate over a width-12 Goldilocks state. Original Poseidon uses rate=8/capacity=4, which is also the standard Poseidon2 Goldilocks configuration (e.g. Plonky3); this fork's Poseidon2 departs from that norm and uses rate=4/capacity=8 instead. The halved rate roughly doubles the constraint cost for application circuits, while the doubled capacity provides additional algebraic security margin for long-lived on-chain values such as nullifiers and vote commitments. The system's security depends on both hash functions remaining secure.

EQ-QNT-ZK-O-05: Verifier Crate Duplicates Gate and Hash Logic from the Prover

The `verifier` crate is a near-verbatim copy of the corresponding `plonky2` (prover) modules, with proving-specific code (`eval_unfiltered_circuit`, witness generators, serialisation) stripped out. What remains — `eval_unfiltered`, gate parameter structs, helper functions — is duplicated across every gate:

Gate	Prover	Verifier	Shared
`poseidon2`	753	354	341
`coset_interpolation`	876	392	386
`poseidon`	653	290	283
`random_access`	543	261	255
`poseidon2_mds`	351	186	179
`exponentiation`	436	183	177

poseidon2_int_mix	305	171	164
lookup_table	264	150	146
poseidon_mds	304	143	135
reducing_extension	269	136	128
arithmetic_base	279	131	127
reducing	274	133	127
base_sum	255	130	124
arithmetic_extension	274	118	112
lookup	247	133	129
multiplication_extension	241	107	102
constant	159	107	103
public_input	136	92	89
noop	92	55	52
Gates total	**7 131**	**3 472**	**3 259**

The "Shared" column counts lines that appear identically in both files (via `diff`).

Free functions and structs duplicated outside the gate layer
Beyond gates, the `hash/` and `plonk/` modules duplicate free functions, struct definitions, and verification logic that does not involve the `Gate`/`VerificationGate` trait at all:

File	Verifier	Shared
plonk/vanishing_poly.rs	775	718
hash/batch_merkle_tree.rs	347	328
plonk/circuit_data.rs	323	301
plonk/proof.rs	305	297
plonk/get_challenges.rs	258	246
hash/poseidon2.rs	150	148
plonk/verifier.rs	118	116
plonk/validate_shape.rs	72	72
hash/path_compression.rs	59	49
Non-gate total	**2 407**	**2 275**

These are ordinary functions and data types — `eval_vanishing_poly`, `check_partial_products`, `CommonCircuitData` fields, proof structs, Merkle-tree hashing — that the prover could import from

the verifier with no trait refactoring required. The prover's copies add `_circuit` variants (`eval_vanishing_poly_circuit`, `check_lookup_constraints_circuit`, etc.) that would remain prover-only.

Together with the gate layer, the total duplicated logic is roughly 5 500 lines.

The duplication is a consequence of how plonky2 represents constraints. In plonky2, each gate's `eval_unfiltered` is imperative Rust: loops, conditionals, and method calls on field-element references that compute constraint values directly. There is no intermediate representation that a second consumer could reinterpret — the only way to get the same constraint logic into the verifier crate is to copy the code.

Other proof systems avoid this by design. Halo2′s gates are defined as symbolic `Expression<F>` trees (sums, products, column queries) that are stored once in the verification key. The prover evaluates those expressions against witness values; the verifier evaluates the same expressions at a random challenge point. Because the constraint is an AST rather than executable code, a single definition serves both roles without duplication [2], [3]. Bellman [4] and Arkworks [5] take a different route: the circuit implements a `synthesize` method generic over a `ConstraintSystem` trait, and different backends (keygen, prover, test harness) supply different implementations of that trait. The verifier never re-runs `synthesize` at all — it works from the verification key alone.

None of these are drop-in solutions for plonky2. They were designed from the ground up around a "define constraints once, interpret many ways" abstraction. Plonky2 was not [6], and the other plonky2 forks (including wormhole-foundation/plonky2-near [7]) sidestep the problem by staying monolithic. The Quantus fork's separate verifier crate is a reasonable way to isolate the `no_std`/WASM-safe subset, and copy-paste was the expedient path given the inherited architecture.

The practical risk is drift: if a bug fix or optimisation is applied to one copy and not the other, the prover and verifier silently diverge. At 5 600 lines of shared logic, keeping the two in sync by hand is error-prone.

Dead prover code carried into the verifier

Some prover-only functions were copied into the verifier and silenced with `#[allow(dead_code)]` rather than removed. In `vanishing_poly.rs`, `eval_vanishing_poly_base_batch` (batch base-field constraint evaluation over the polynomial domain) is never called in the verifier — its own doc-comment says "Used by the prover". It in turn calls `evaluate_gate_constraints_base_batch` and `check_lookup_constraints_batch`, which are also dead. In `hash/poseidon2.rs`, `hash_no_pad_bytes` is likewise unused; the wormhole circuits import the prover's copy, not the verifier's.

None of these functions are reachable from the verifier's public API.

Recommendation
Both categories of duplication can be resolved by having the prover depend on the verifier crate. The verifier is the minimal `no_std`/WASM-safe subset; the prover is a strict superset. The work falls into two tiers of difficulty:

Tier 1 — Import shared functions and types. The ~2 300 lines of duplicated free functions and structs (`vanishing_poly`, `circuit_data`, `proof`, `get_challenges`, Merkle-tree hashing) can be imported directly from the verifier once the prover lists it as a dependency. The prover keeps its `_circuit` variants as local additions. This requires no trait changes, only adjusting import paths and removing the prover's redundant copies. The dead `#[allow(dead_code)]` functions in the verifier should be removed at the same time.

Tier 2 — Gate trait refactor. The ~3 300 lines of duplicated gate code require restructuring the trait hierarchy. Gate structs, parameter types, MDS helpers, and `eval_unfiltered` would be defined once in the verifier under the existing `VerificationGate` trait. The prover would import them and extend the gate types with proving-specific methods (`eval_unfiltered_circuit`, witness generators) via a `Gate: VerificationGate` supertrait — implementing a local trait on a foreign type is permitted by Rust's orphan rules.

This is a large refactor. Plonky2′s recursive verification requires every gate to implement `eval_unfiltered_circuit` (expressing constraints symbolically via `CircuitBuilder`), which bundles prover-side types into the `Gate` trait. Splitting that into a `VerificationGate` supertrait in the verifier and a `Gate` extension in the prover would touch the gate dispatch machinery, `CommonCircuitData`, trait object handling, and every gate implementation across both crates.

EQ-QNT-ZK-O-06: Protocol-Level Constants Duplicated across Crates without a Single Source of Truth

Several protocol-level constants are independently defined in more than one crate. All copies agree today, but a future edit to one without updating the others would silently break proof verification. Poseidon2-specific duplications are covered separately in EQ-QNT-ZK-O-03; the remaining cases are listed below.

Lookup challenge parameters
The number of random coins per lookup interaction and the enum mapping coin names to indices are duplicated between the prover and verifier:

Constant	Canonical (plonky2)	Verifier copies
`NUM_COINS_LOOKUP = 4`	`plonk/circuit_builder.rs`	`plonk/vanishing_poly.rs`, `plonk/get_challenges.rs` (function-local)

`LookupChallenges` enum	`plonk/circuit_builder.rs`	`plonk/vanishing_poly.rs`

`NUM_COINS_LOOKUP` controls how many Fiat-Shamir challenges are squeezed from the sponge during `get_challenges`: `NUM_COINS_LOOKUP * num_challenges` elements are derived, of which the first two batches are reused from `plonk_betas` and `plonk_gammas` and the rest are freshly sampled. The `LookupChallenges` enum (`ChallengeA = 0`, `ChallengeB = 1`, `ChallengeAlpha = 2`, `ChallengeDelta = 3`) maps each coin to its role in the lookup argument. If either constant diverges, the Fiat-Shamir transcript desynchronises and valid proofs are rejected.

Polynomial blinding salt size

`SALT_SIZE` (the number of random field elements appended to blinded polynomial evaluations) is defined in `core` and re-exported through `plonky2/plonk/plonk_common.rs` — but the FRI oracle defines its own independent copy:

Location	Definition
`core/src/plonk_common.rs:10`	`pub const SALT_SIZE: usize = 4` (canonical)
`plonky2/src/fri/oracle.rs:26`	`pub const SALT_SIZE: usize = 4` (independent copy)

The FRI oracle's `PolynomialBatch` uses its local copy when computing the number of blinding columns. A mismatch would cause the prover to commit polynomials of a different degree than the verifier expects, breaking the FRI degree bound check. Because `SALT_SIZE` controls the zero-knowledge property (capacity elements that mask committed polynomials), an undersized salt could also weaken privacy guarantees.

Keccak sponge parameters

The Keccak-based sponge hasher constants are defined independently in `core` and `plonky2`:

Constant	core/src/keccak.rs	plonky2/src/hash/keccak.rs
`SPONGE_RATE = 8`	line 12	line 13
`SPONGE_CAPACITY = 4`	line 13	line 14
`SPONGE_WIDTH = 12`	line 14	line 15

The `plonky2` copy does not import from `core`; it redeclares identical values. A mismatch would produce different Merkle roots and Fiat-Shamir challenges for any circuit using the Keccak configuration.

All of these are specific instances of the broader duplication described in EQ-QNT-ZK-O-05.

Recommendation
Move `NUM_COINS_LOOKUP`, the `LookupChallenges` enum, and the Keccak sponge parameters to `qp-plonky2-core`. Remove the independent `SALT_SIZE` in `fri/oracle.rs` and import the existing definition from `core`. As a stopgap, add comments to each copy referencing the canonical location.

6. ZK TRIE AND STATE MACHINE

`zk-trie` is a fork of Substrate's `sp-trie` that pads every trie node component to 8-byte boundaries for processing in plonky2 circuits over the Goldilocks field. `zk-state-machine` is a thin companion fork of `sp-state-machine` that points its trie dependency at `zk-trie` and disables compact proofs.

The review covers all changes from the fork points (`f945800` for `zk-trie`, `83fb5c7` for `zk-state-machine`) through the pinned submodule heads. The felt-alignment encoding is structurally correct: the invariant that every encoded node is a multiple of 8 bytes holds across all node types, and `StorageProof`, used by the ZK circuit pipeline, is unchanged from upstream. No exploitable issues were identified in the encoding changes.

EQ-QNT-TRIE-O-02: Compact Proof API Simplified to Passthrough

Substrate's `CompactProof` is a space-saving re-encoding of a `StorageProof`. It strips redundant child hash references from trie nodes, since those hashes can be recomputed from the child node data that is also present in the proof. This optimization is used in parachain validation where proof bandwidth matters.

The compact encoding depends on `trie_db::encode_compact`, which parses trie nodes internally using the upstream Substrate codec. The felt-aligned encoding changes the node format entirely – 8-byte headers, different partial key layouts, different value encoding , so the upstream compact codec cannot parse zk-trie nodes.

Rather than rewriting the compact codec for the new format, `zk-trie` simplifies `CompactProof` to a transparent wrapper around `StorageProof`. The conversion in both directions is a no-op copy of the node list:

src/storage_proof.rs:134-141

```
134 |     /// Encode as a compact proof with default trie layout.
135 |     pub fn into_compact_proof<H: Hasher>(
136 |         self,
137 |         _root: H::Out,
138 |     ) -> Result<CompactProof, crate::CompactProofError<H::Out, crate::Error<H::Out>>> {
```

```
        // Since CompactProof now wraps StorageProof, just create a CompactProof directly
        Ok(CompactProof::from_storage_proof(self))
    }
```

src/storage_proof.rs:205-218

```
    /// Decode to a full storage_proof.
    pub fn to_storage_proof<H: Hasher>(
        &self,
        expected_root: Option<&H::Out>,
    ) -> Result<(StorageProof, H::Out), crate::CompactProofError<H::Out, crate::Error<H::Out>>>
    {
        // Since CompactProof now just wraps StorageProof data, convert back to StorageProof
        let storage_proof = StorageProof::new(self.encoded_nodes.clone());
        let result = match expected_root {
            Some(root) => Ok((storage_proof, *root)),
            None => Ok((storage_proof, H::Out::default())),
        };
        result
    }
```

`to_storage_proof` accepts an `expected_root` parameter for API compatibility but does not verify it; it returns whatever root is passed in. The upstream implementation calls `decode_compact` to reconstruct the trie and verify the root against the proof, but since there is no compact encoding to decode, that verification is not applicable.

The `encode_compact` and `decode_compact` functions in `trie_codec.rs` still exist but are orphaned: no code path reaches them. Quantus does not use parachain validation or any other consumer of compact proofs, and the ZK circuit pipeline operates on `StorageProof` directly.

Recommendation

Remove the `CompactProof` type and the orphaned `encode_compact/decode_compact` functions from `zk-trie`. The type retains the upstream function signatures but silently drops root verification. A caller familiar with the upstream API would have no compile-time indication that `to_storage_proof` no longer validates the `expected_root`. Since nothing in the Quantus codebase uses `CompactProof`, removing it eliminates this footgun without breaking any consumer.

7. Experiments

A. Miner Volume Fee Misdirection

To confirm EQ-QNT-WORMHOLE-F-03, a valid aggregated proof was submitted to a devnet node and included in block `#11247`. The consensus engine placed a `PreRuntime(pow_)` digest entry in that block with this 32-byte preimage:

```
0x6815f56f0218be47c2075a161c4ae8a9f574d4476ef7f64c0a11d3779376d439
```

The mining-rewards and wormhole pallets derive different recipient addresses from the same preimage:

Pallet	Derivation	Address (hex)
Mining-rewards (correct)	Poseidon2(preimage) then decode	`5c60321214ac4e63157164040322573e3bd871fc176bdc57170eeabc3e889764`
Wormhole (bug)	decode(preimage) directly	`6815f56f0218be47c2075a161c4ae8a9f574d4476ef7f64c0a11d3779376d439`

Balance changes between blocks `#11246` (before) and `#11248` (after) show where each pallet sent funds:

Address	Role	Balance delta
`6815f56f…6d439` (raw preimage bytes)	Wormhole bug target	+49,989,989,990 planck (0.050 DEV)
`5c603212…9764` (Poseidon2 hash)	Correct miner address	+547,710,763,035 planck (0.548 DEV)

The mining-rewards pallet correctly credited the miner `~0.548 DEV` in block rewards. The wormhole pallet minted `~0.050 DEV` in volume fees to the raw-preimage address — an account the miner does not control.

B. Garbage Proof Block Saturation

The question for EQ-QNT-WORMHOLE-F-02 is how much effort block saturation actually takes. Garbage payloads were sent to `verify_aggregated_proof` on a devnet node from a single unoptimised client over one WebSocket connection. No account or balance was needed.

The declared weight is a placeholder (see EQ-QNT-WORMHOLE-R-09). In picoseconds (the unit of `ref_time`), with RocksDb overhead included:

$$w_{\text{base}} = 115\,744\,000\,000 \text{ ps} = 115.7 \text{ ms}$$

$$w_{\text{storage}} = 32 \times 25\,000\,000 \text{ ps} + 32 \times 100\,000\,000 \text{ ps} = 4\,000\,000\,000 \text{ ps} = 4.0 \text{ ms}$$

$$w_{\text{total}} = w_{\text{base}} + w_{\text{storage}} = 119\,744\,000\,000 \text{ ps} \approx 119.7 \text{ ms}$$

$$\left\lfloor 4500 \frac{\text{ms}}{119.7 \text{ ms}} \right\rfloor = 37 \text{ proofs per block}$$

`System::BlockWeight` queried on a block from the run:

Dispatch class	ref_time consumed (ps)	Normal-quota fill
Normal (garbage proofs)	4 434 529 809 000	98.55% (2.66% × 37)
Mandatory (inherents)	3 954 574 146	0.09%
Total	**4 438 484 383 146**	**98.63%**

`4 434 529 809 000 / 119 744 000 000 = 37.0`: exactly 37 garbage proofs, leaving 1.45% headroom that is too small for any further extrinsic. Six blocks were produced during the 60-second run; none contained user transactions.

The client ran at 8.7 submissions/s. Block drain is 3.7/s (37 proofs per 10 s block), so the pool grew throughout and never stabilised. One connection is enough.

8. CLASSIFICATIONS

We use standard CVSS severity classifications:

- **Critical**
- **High**
- **Medium**
- **Low**
- **None (Information)**

Generally, we mark findings Critical if an exploit or bug may cause a complete system compromise, outage, data loss or regulatory impact. This could be through key recovery, signature forgery or other protocol insecurities. The issues should be fixed with highest priority.

We mark findings as High if the exploit may have a significant security impact in degradation of service or exposure of sensitive information, or if the system is volatile under specific mis-use. This covers attacks under realistic assumptions and resource bounds that must be fixed.

Findings marked as Medium causes limited impact, for example minor data exposure or isolation to non-critical components. This also covers public APIs susceptible to mis-use.

The Low findings cause low or negligible security impact. This covers theoretical or impractical conditions, but highlights weaknesses such as weak entropy, undocumented skews in randomness, or deviation from best practices.

The Informative findings contain no vulnerabilities and cover general proposals and comments.

9. CONCLUSION

The review identified 35 findings across the wormhole pallet, mining-rewards pallet, Poseidon2 hash implementation, ZK circuits, and supporting infrastructure. The cryptographic primitives are correctly implemented and the Poseidon2 permutation conforms to the published specification.

We recommend prioritising two fixes before mainnet deployment:

0. **Harden the unsigned proof endpoint against resource exhaustion.** EQ-QNT-WORMHOLE-F-02

1. **Correct miner fee address derivation to prevent silent token burn.** EQ-QNT-WORMHOLE-F-03

The remaining findings are lower severity and relate to code maintainability, documentation, and test coverage. None pose an immediate security risk, but pose a maintainability burden, if unadressed.

10. REFERENCES

[1] "Not So Smart Contracts: Substrate — validate_unsigned." [Online]. Available: https://secure-contracts.com/not-so-smart-contracts/substrate/validate_unsigned/

[2] Electric Coin Company, "halo2." [Online]. Available: https://github.com/zcash/halo2

[3] "PLONKish Arithmetization — The halo2 Book." [Online]. Available: https://zcash.github.io/halo2/concepts/arithmetization.html

[4] zkcrypto, "bellman." [Online]. Available: https://github.com/zkcrypto/bellman

[5] arkworks, "ark-relations." [Online]. Available: https://github.com/arkworks-rs/r1cs-std

[6] Polygon Zero, "Plonky2: A Deep Dive." [Online]. Available: https://polygon.technology/blog/plonky2-a-deep-dive

[7] Wormhole Foundation, "plonky2-near." [Online]. Available: https://github.com/wormhole-foundation/plonky2-near

[8] L. Grassi, D. Khovratovich, and M. Schofnegger, "Poseidon2: A Faster Version of the Poseidon Hash Function," 2023, [Online]. Available: https://eprint.iacr.org/2023/323

[9] "p3-poseidon2." [Online]. Available: https://crates.io/crates/p3-poseidon2

[10] "qp-poseidon core library." [Online]. Available: https://github.com/Quantus-Network/qp-poseidon/blob/main/core/src/lib.rs

[11] "Plonky3 Poseidon2 Round Numbers." [Online]. Available: https://github.com/0xPolygonZero/Plonky3/blob/main/poseidon2/src/round_numbers.rs

[12] "Plonky3 Goldilocks Poseidon2." [Online]. Available: https://github.com/0xPolygonZero/Plonky3/blob/main/goldilocks/src/poseidon2.rs

[13] "qp-poseidon-constants." [Online]. Available: https://crates.io/crates/qp-poseidon-constants

[14] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "On the Indifferentiability of the Sponge Construction," in *EUROCRYPT 2008*, 2008.

[15] D. Kales and G. Zaverucha, "Forgery Attack on Parity," 2023, [Online]. Available: https://eprint.iacr.org/2023/537

[16] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A New Hash Function for Zero-Knowledge Proof Systems," 2019, [Online]. Available: https://eprint.iacr.org/2019/458.pdf

11. APPENDIX A: Poseidon2 Permutation Parameters

	Value	Reference
Field	Goldilocks: $p = 2^{64} - 2^{32} + 1$	[8]
Width (t)	12	[9]
Rate	4	[10]
Capacity	8	$t - \text{rate}$
Output	4 elements (256 bits)	[10]
Full rounds (R_F)	8 (4 initial + 4 terminal)	[11]
Partial rounds (R_P)	22	[11]
External 4×4 matrix	Circulant $[2, 3, 1, 1]$ (`MDSMat4`)	[8]
Internal diagonal ($M_I - I$)	See below	[12]
Round constant seed	`0x3141592653589793` (digits of π)	[13]
Round constant PRNG	ChaCha20	[13]

Internal diagonal. The internal round mixing uses $y_i = d_i \cdot x_i + \sum_j x_j$ where d_i are the diagonal entries of $M_I - I$ (`MATRIX_DIAG_12_GOLDILOCKS` in Plonky3):

```
0xc3b6c08e23ba9300  0xd84b5de94a324fb6  0x0d0c371c5b35b84f
0x7964f570e7188037  0x5daf18bbd996604b  0x6743bc47b9595257
0x5528b9362c59bb70  0xac45e25b7127b68b  0xa2077d7dfbb606b5
0xf3faac6faee378ae  0x0c6388b51545e883  0xd27dbb6944917b60
```

S-box. The S-box is x^7. The exponent $d = 7$ is valid because $\gcd(7, p - 1) = 1$, ensuring the power map is a permutation over the Goldilocks field.

Security margin. The sponge security scales with $p^{c/2}$ assuming an ideal random permutation [14]. With the 64-bit Goldilocks prime and capacity $c = 8$, the generic bound is $(2^{64})^4 = 2^{256}$ bits

of classical collision resistance. The round numbers $(R_F, R_P) = (8, 22)$ are separately chosen so that six independent constraint families — statistical, interpolation, Gröbner basis (three variants), and an additional bound from Kales and Zaverucha [15] — each require at least 2^{128} operations to break the permutation [11], [16]. The security analysis is inherited from the original Poseidon paper, as Poseidon2 changes only the linear layer and preserves the same round number framework. This is the binding constraint: the overall security is 128 bits, bottlenecked by algebraic resistance of the permutation rather than by the sponge capacity.

12. APPENDIX B: Wormhole Nullifier Storage Growth Estimate

The `UsedNullifiers` storage map is declared in the wormhole pallet as:

```
StorageMap<_, Blake2_128Concat, [u8; 32], bool, ValueQuery>
```

Per-entry key-value size

Each entry in a Substrate `Blake2_128Concat` `StorageMap` occupies a full storage key plus the SCALE-encoded value:

Component	Bytes	Notes
`twox_128(pallet_name)`	16	Shared prefix for the pallet
`twox_128("UsedNullifiers")`	16	Shared prefix for the storage item
`Blake2_128(nullifier)`	16	Hash portion of `Blake2_128Concat`
raw nullifier key (`[u8; 32]`)	32	Appended by the `Concat` variant
SCALE `bool` value	1	
Total per entry (raw)	**81**	

Each entry also materialises as a trie leaf node in the state backend (Substrate's Patricia-Merkle trie). The leaf node is stored as a separate RocksDB key-value pair: its hash (32 bytes) as the RocksDB key, plus the encoded node data (partial nibble path and SCALE value, roughly 40–50 bytes). Amortised across a dense, uniformly-distributed key space, trie node overhead adds approximately 60–80 bytes per entry, bringing the realistic per-entry cost to around **150 bytes**.

Annual growth calculation

The chain targets a 12-second block time (`TARGET_BLOCK_TIME_MS = 12_000`, `runtime/src/lib.rs`). Each aggregated proof carries exactly $N = 8$ nullifiers (the configured `num_leaf_proofs` value in the wormhole aggregator), all of which are written unconditionally. At 37 aggregated proofs per block — the maximum permitted by the currently configured weight — every fully-saturated block writes:

$$37 \times 8 = 296 \text{ nullifiers}$$

Blocks per year:

$$\left\lfloor \frac{365.25 \times 24 \times 3600 \ \ \mathrm{s}}{12 \ \ \mathrm{s}} \right\rfloor \approx 2{,}629{,}800$$

Nullifier entries per year at full saturation:

$$296 \times 2{,}629{,}800 \approx 7.79 \times 10^8$$

Storage consumed per year:

Basis	GiB/year
Raw key + value (81 bytes)	$\frac{7.79 \times 10^8 \times 81}{1024^3} \approx 59$
Including trie overhead (150 bytes)	$\frac{7.79 \times 10^8 \times 150}{1024^3} \approx 109$

These figures are upper bounds under continuous, full-block saturation. Actual growth is proportional to batch throughput. The `UsedNullifiers` map is never pruned; it grows monotonically for the lifetime of the chain.